
ML - Machine Learning

Support Vector Machines

Riccardo Berta

2026.04.21

Contents

1	Support Vector Machines	1
1.1	Separating Hyperplane	2
1.1.1	Margin	6
1.1.2	Maximum margin hyperplane	7
1.1.3	Multiclass classification	10
1.2	Lagrangian method	10
1.2.1	Equality constraints	11
1.2.2	Multiple equality and inequality constraints	19
1.2.3	Dual formulation	21
1.3	Optimal margin classifier	24
1.3.1	Separable case	24
1.3.2	Non-separable case	30
1.4	Nonlinear Classification	39
1.4.1	Kernels	39
1.4.2	Kernelized SVM	40
1.5	Regression	44
1.5.1	Linear regression	45
1.5.2	Kernelized Regression	53
1.6	Curse of dimensionality	55
1.6.1	Generalization bounds	56
1.6.2	Advanced techniques	57
1.7	Exercise	58

1 Support Vector Machines

A Support Vector Machine (SVM) is a supervised learning method originally introduced for classification, and later extended to regression and other tasks. At its core, an SVM adopts a **geometric view of learning**: instead of simply separating the data, it seeks a decision boundary that does so with the **largest possible margin**. In the simplest setting, the decision boundary is a hyperplane in the feature space. When the data is linearly separable, many such hyperplanes exist. SVMs select the one that maximizes the **margin**, that is, the **minimum distance between the hyperplane and the training points**. This choice is not arbitrary: a larger margin implies that the classifier is less sensitive to small perturbations of the data, and therefore more likely to generalize well to unseen examples.

A key consequence of this formulation is that the solution depends only on a subset of the training data: the points that lie closest to the decision boundary. These points are called **support vectors**, because they alone determine the position and orientation of the hyperplane. All other training examples, as long as they remain sufficiently far from the boundary, do not directly influence the solution. A direct implication of this is that SVMs tend to produce **compact and effective representations**, especially in high-dimensional settings where the number of features is large relative to the number of samples.

One of the most powerful aspects of SVMs is that this geometric idea extends naturally to **non-linear problems**. When the data cannot be separated by a linear hyperplane in the original space, SVMs rely on the **kernel trick** to implicitly map the data into a higher-dimensional feature space, where a linear separation may exist. Crucially, this mapping is never computed explicitly: instead, the algorithm operates entirely through kernel functions, which evaluate inner products in the transformed space. This allows SVMs to learn highly nonlinear decision boundaries while maintaining a tractable optimization problem.

Overall, SVMs combine a strong geometric intuition with a rigorous theoretical foundation based on margin maximization and convex optimization. This combination explains both their robustness and their lasting relevance in the landscape of machine learning methods.

1.1 Separating Hyperplane

Imagine we are given a training dataset consisting of m labeled examples:

$$L = \{(x_i, y_i)\}_{i=1}^m, \quad x_i \in \mathbb{R}^D, \quad y_i \in \{-1, 1\}$$

Each vector represents a point in a D -dimensional feature space, while the label indicates to which of the two classes the point belongs. We assume that the dataset is **linearly separable**, meaning that it is possible to draw a hyperplane that perfectly separates the two classes. Geometrically, a hyperplane is a $(D-1)$ -dimensional surface defined by:

$$H = \{x : w^T x + b = 0\}$$

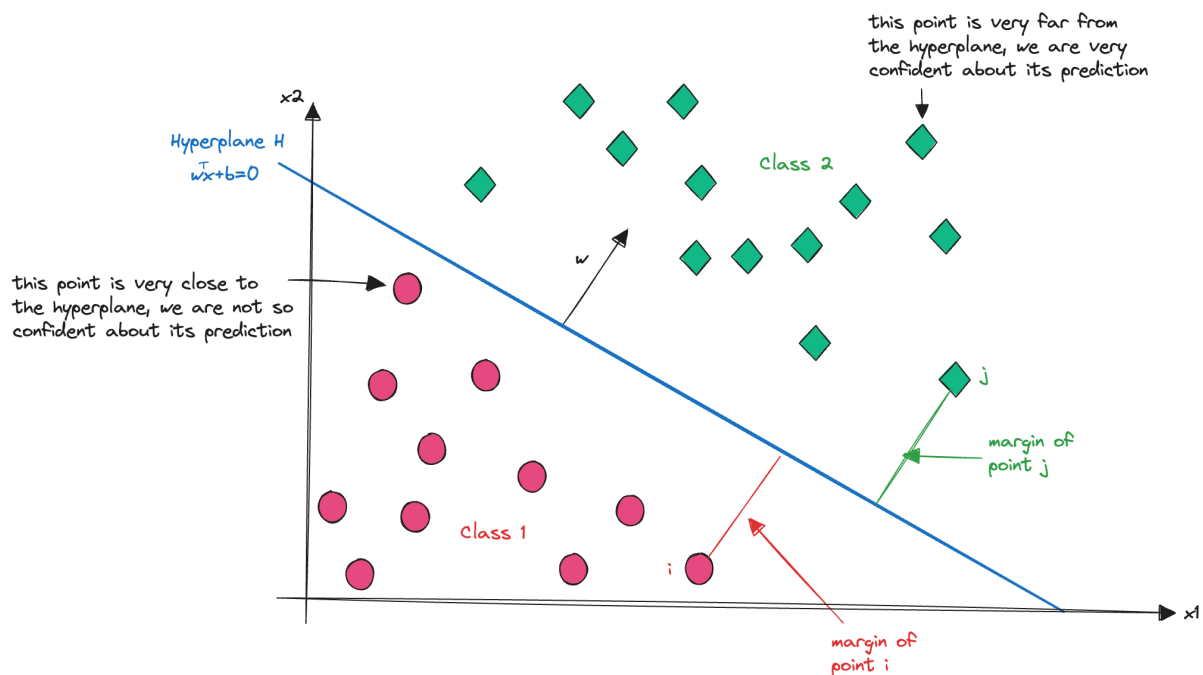
where w is a vector orthogonal to the hyperplane and b is the bias (or intercept) term. The vector w determines the **orientation** of the hyperplane, while b determines its **position** in space. This hyperplane naturally defines a linear classifier:

$$h_{w,b}(x) = g(w^T x + b)$$

where $g()$ is the sign function:

$$g(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

In other words, the classifier determines the label of a point by **checking on which side of the hyperplane the point lies**. Points located on one side of the separating surface are assigned to class +1, while points located on the opposite side are assigned to class -1. Notice that, unlike the usual linear model notation that collects all parameters into a single vector, here we explicitly separate the parameters into (w, b) . This distinction highlights the geometric roles of the two quantities: the vector w determines the orientation of the separating hyperplane (it is perpendicular to it), while b determines its position in space, shifting the hyperplane along the direction of w .



As a concrete example, we can consider the Iris dataset, using just two classes, versicolor and setosa and two features, petal length and petal width:

```
import numpy as np
from sklearn import datasets
iris = datasets.load_iris()

X = iris["data"][:, (2, 3)]
y = iris["target"]

setosa_or_versicolor = (y == 0) | (y == 1)
X = X[setosa_or_versicolor]
y = y[setosa_or_versicolor]

# use {-1, +1} as in the math
```

```
y = np.where(y == 0, -1, 1).astype(float)
```

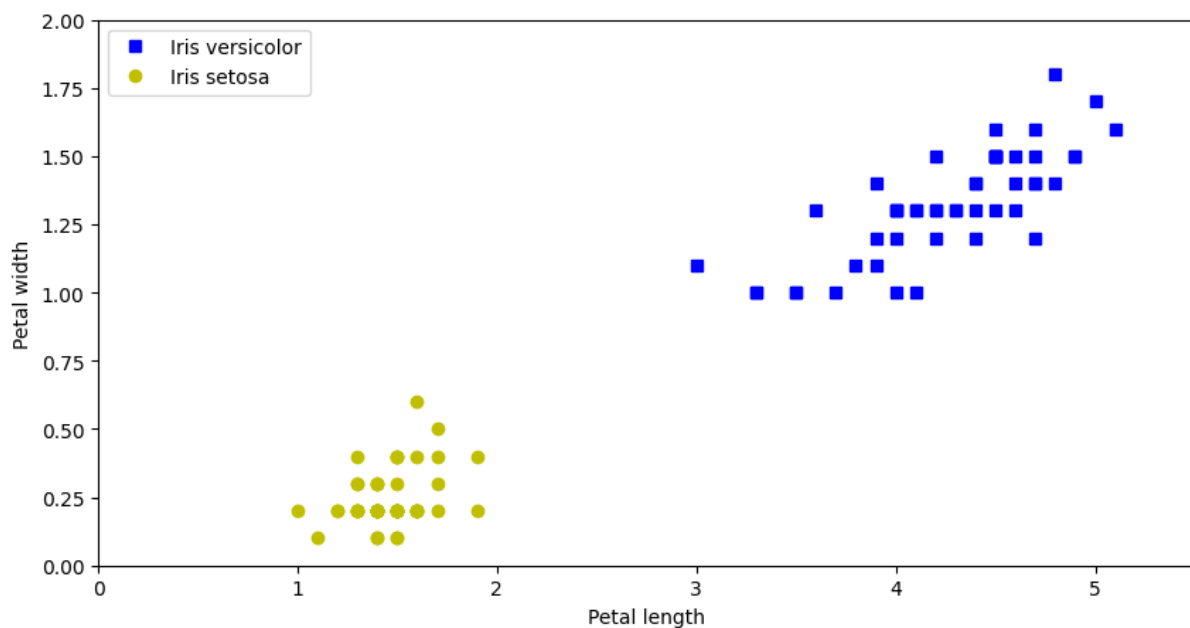
The two classes of the example are clearly linearly separable:

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(X[:, 0][y==1], X[:, 1][y==1], "bs", label="Iris versicolor")
ax.plot(X[:, 0][y==-1], X[:, 1][y==-1], "yo", label="Iris setosa")

ax.set_xlabel("Petal length")
ax.set_ylabel("Petal width")
ax.legend(loc="upper left")
ax.axis([0, 5.5, 0, 2])
plt.show()
```



We can find many different hyperplanes (in that case simple lines) that separate them, just by visual inspection:

```
x0 = np.linspace(0, 5.5, 200)
```

```
pred_1 = 5*x0 - 20
pred_2 = x0 - 1.8
pred_3 = 0.1 * x0 + 0.5
```

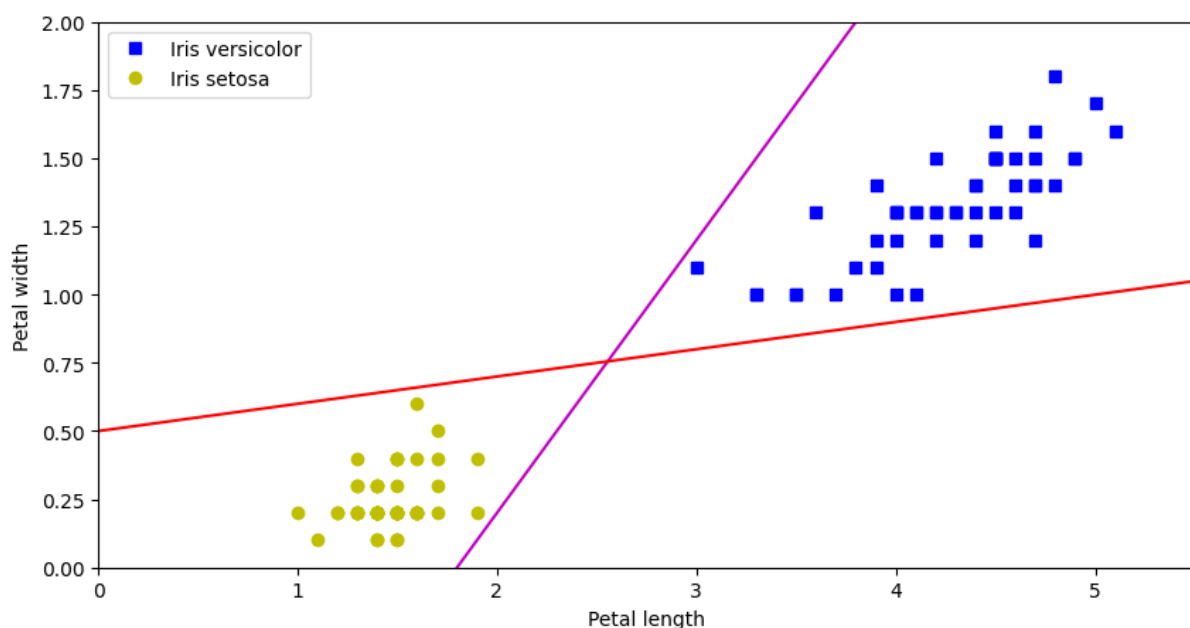
Plotting the selected lines we can see that they all separate the two classes:

```
fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(x0, pred_2, "m-")
ax.plot(x0, pred_3, "r-")

ax.plot(X[:, 0][y==1], X[:, 1][y==1], "bs", label="Iris versicolor")
ax.plot(X[:, 0][y==0], X[:, 1][y==0], "yo", label="Iris setosa")

ax.set_xlabel("Petal length")
ax.set_ylabel("Petal width")
ax.legend(loc="upper left")
ax.axis([0, 5.5, 0, 2])
plt.show()
```

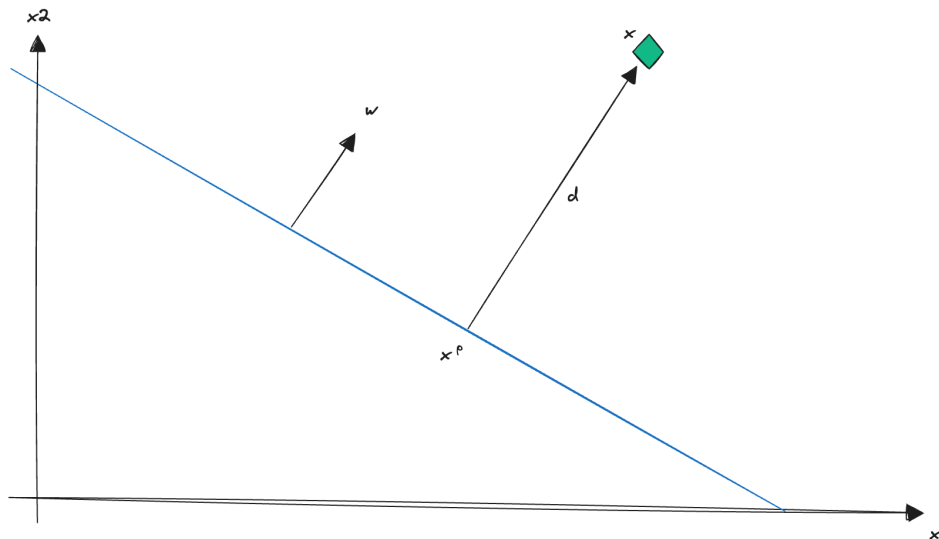


However, some of these hyperplanes are more robust than others. Now consider how points are positioned relative to the hyperplane. If a point lies far away from the decision boundary, it is clearly on one side of the space, and the classifier can assign its label with **high confidence**. In contrast, if a point lies very close to the hyperplane, its position is more **ambiguous**: even a

small perturbation of the data could move it to the other side of the boundary and change the predicted class. This observation naturally suggests that **not all correctly classified points are equally reliable**: points that are farther from the separating surface are classified with greater confidence than those that lie close to it. This observation motivates the notion of **margin**, which measures **how far a point lies from the separating hyperplane**.

1.1.1 Margin

To understand the core idea behind SVMs, we first need a precise notion of **margin**. Geometrically, the margin is the **shortest distance between a point and the separating hyperplane**. This shortest distance is not arbitrary: it is achieved by moving from the point toward the hyperplane along the **direction perpendicular** to it, which is the direction of the normal vector, as illustrated in the figure below:



We can compute this distance using a simple geometric idea. Take a point in the feature space and imagine "dropping" it perpendicularly onto the hyperplane. The closest point on the hyperplane is its projection, and the segment connecting the point to this projection represents the shortest path to the hyperplane. The vector connecting the point to its projection is:

$$d = x - x^p$$

and the distance we are looking for is simply the length of this vector:

$$|d| = \sqrt{d_1^2 + d_2^2 + \dots + d_D^2} = \sqrt{d^T d}$$

Now, the key geometric observation is that the shortest path to a hyperplane is always orthogonal to it. This means that the direction of this vector must coincide with the direction normal to

the hyperplane. Therefore there exists a scalar such that:

$$d = \alpha w$$

To determine this scalar, we can use the fact that the projection lies exactly on the hyperplane, and therefore it satisfies:

$$w^T x^p + b = 0$$

Substituting the expression for the projection into this condition gives:

$$w^T (x - \alpha w) + b = 0$$

from which we obtain:

$$\alpha = \frac{w^T x + b}{w^T w}$$

We can compute the distance between the point and the hyperplane in terms of the normal vector and the bias:

$$|d| = \sqrt{d^T d} = \sqrt{\alpha^2 w^T w} = \sqrt{\left(\frac{w^T x + b}{w^T w}\right)^2 w^T w} = \frac{|w^T x + b|}{\sqrt{w^T w}} = \frac{|w^T x + b|}{|w|}$$

This final expression has a clear geometric interpretation: the numerator measures how far the point is from the hyperplane along the direction of the normal vector. However, this is not yet a true distance: its value depends on **how the normal vector is scaled**. The denominator corrects this. It removes the dependence on the scale of w by normalizing, and providing a true Euclidean distance measured in the feature space:

$$\gamma^{(i)}(w, b) = \frac{|w^T x^{(i)} + b|}{|w|}$$

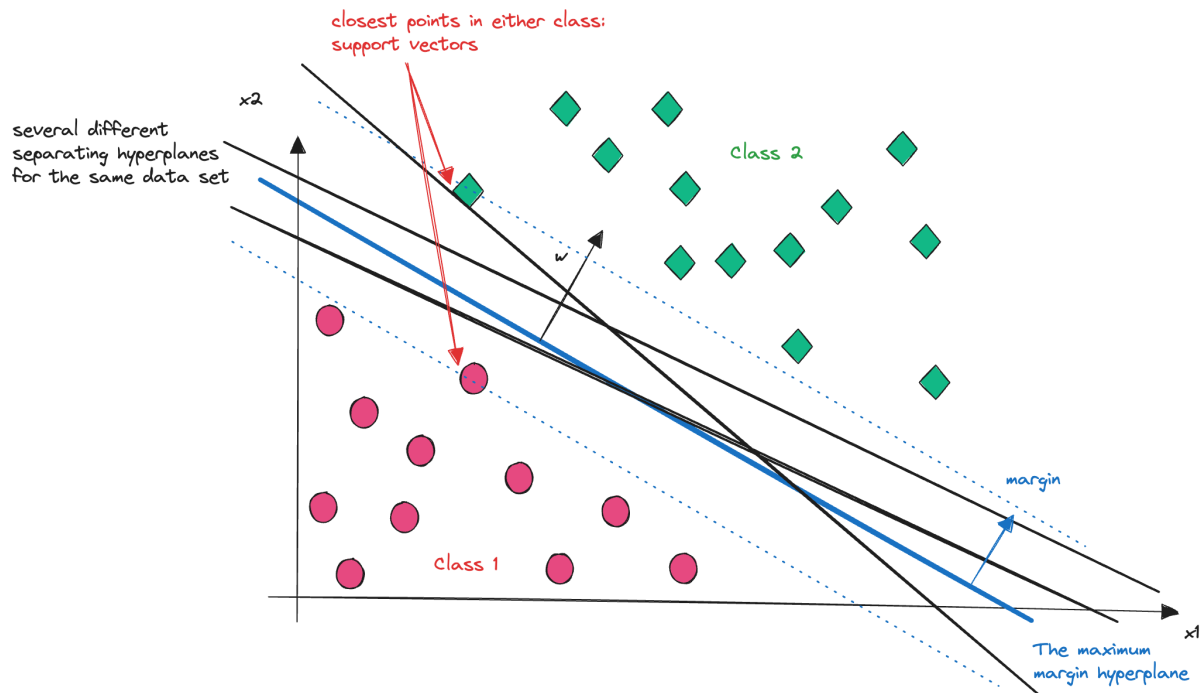
Intuitively, **the farther a point is from the decision boundary, the more confident the classifier is in its prediction**. Points that lie very close to the hyperplane are instead more uncertain: even a small perturbation could move them to the other side and change their label. For this reason, **not all correctly classified points are equally informative, the most critical ones are those closest to the boundary**. Given the entire training dataset, the quality of a hyperplane is therefore **determined by its worst-case scenario****, that is, by the point that lies closest to it. This leads to the definition of the margin of the hyperplane as the smallest margin among all training points:

$$\gamma(w, b) = \min_{i=1, \dots, m} \gamma^{(i)}(w, b)$$

1.1.2 Maximum margin hyperplane

We can have **many possible hyperplanes that could separate the classes**, however, some of them may be **more robust** than others. The quality of the hyperplane is determined by the

point that lies closest to it. If that closest point is still far away, then the separator is robust. This leads to the central idea of SVMs: **choose the hyperplane that maximizes this minimum margin**, thereby placing the decision boundary as far as possible from the training examples:



Formally, we would like to find a hyperplane that is as far as possible from all training points (maximizing the margin), while ensuring that all training points are correctly classified (satisfying the classification constraints). This can be expressed as the following optimization problem:

$$\begin{cases} \max_{w,b} \gamma(w,b) \\ \text{s.t. } y^{(i)}(w^T x^{(i)} + b) \geq 0 \quad \text{for } i = 1, \dots, m \end{cases}$$

At this stage, the margin has the form

$$\gamma(w,b) = \min_{i=1,\dots,m} \frac{|w^T x^{(i)} + b|}{|w|}$$

and the optimization involves a ratio: the numerator depends on both the normal vector and the bias, while the denominator depends on the norm of the normal vector. This makes the problem difficult to handle directly. However the hyperplane does not change if we scale its parameters. Multiplying both the normal vector and the bias term by the same positive constant leaves the decision boundary exactly where it is. This means that **the representation of the hyperplane is not unique**, and we can exploit this freedom to simplify the problem. In general the normal vector of a hyperplane is selected in order to be a unit vector, but in this particular case, it is more convenient to choose a different value that will lead to a simpler expression for the margin. The

idea is to fix the numerator and impose that **the closest points have distance equal to 1**:

$$\min_{i=1,\dots,m} |w^T x^{(i)} + b| = 1$$

With this normalization, the numerator in the margin expression is fixed, and the margin simplifies to:

$$\gamma(w, b) = \frac{1}{|w|}$$

Now the problem is much simpler: maximizing the margin means making the norm of the normal vector as small as possible. At the same time, the classification constraints become stronger. Instead of just requiring correct classification, we require every point to lie outside the margin boundaries:

$$y^{(i)}(w^T x^{(i)} + b) \geq 1 \quad \text{for } i = 1, \dots, m$$

So the maximum-margin problem can be reformulated as the following optimization problem, where we introduce a multiplicative factor for derivatives convenience:

$$\begin{cases} \min_{w,b} \frac{1}{2} w^T w \\ \text{s.t. } y^{(i)}(w^T x^{(i)} + b) \geq 1 \quad \text{for } i = 1, \dots, m \end{cases}$$

This formulation also gives a beautiful geometric interpretation. Under the chosen scaling, the closest points satisfy:

$$y^{(i)}(w^T x^{(i)} + b) = 1$$

so they lie on one of the two parallel hyperplanes:

$$w^T x + b = 1$$

$$w^T x + b = -1$$

The decision boundary itself lies exactly in the middle:

$$w^T x + b = 0$$

The distance from the central hyperplane to either boundary is:

$$\frac{1}{|w|}$$

so the total width of the margin is:

$$\frac{2}{|w|}$$

This shows that minimizing the norm of the normal vector is equivalent to maximizing the geometric separation between the two classes. By making $|w|$ smaller, we are effectively pushing the two margin boundaries farther apart, and therefore increasing the gap between the classes.

At the optimum, some training points will lie exactly on the margin boundaries. These points satisfy:

$$y^{(i)}(w^T x^{(i)} + b) = 1$$

and are called **support vectors**. They play a crucial role because they are the points that are closest to the decision boundary, and therefore they completely determine its position. Geometrically, the hyperplane is "held in place" by these points. If we were to move one of the support vectors and retrain the model, the optimal hyperplane would change accordingly. In contrast, training points that lie well inside their respective regions (far from the margin) do not directly influence the solution. Small perturbations of these points typically do not affect the hyperplane, unless they move close enough to the boundary to become support vectors themselves.

1.1.3 Multiclass classification

SVMs are inherently **binary classifiers**, they have a decision boundary that separates two classes. Extending to more classes requires a strategy to **decompose** the multiclass problem into a set of binary problems. Two approaches are standard:

One-versus-the-rest (OvR) trains K binary classifiers, one per class. Classifier k treats class k as positive and all other classes as negative. At prediction time, each classifier returns a decision value (the signed distance from the hyperplane), and the class with the **highest decision value** wins:

$$\hat{y} = \arg \max_k (w_k^T x + b_k)$$

One-versus-one (OvO) trains one classifier for every pair of classes. At prediction time each classifier casts a vote and the class with the most votes is selected. OvO trains on smaller subsets (only the two relevant classes) so each individual problem is faster, but the total number of models grows quadratically with K .

1.2 Lagrangian method

The maximum-margin formulation we have just derived leads to a specific type of optimization problem: the **objective function** is quadratic, while the **constraints** are linear inequalities. Problems of this kind fall into the class of **constrained optimization problems**, for which we need appropriate mathematical tools to find the solution. In general, a constrained optimization problem can be written in the following standard form:

$$\begin{cases} \min_x f(x) \\ \text{s.t. } g_i(x) \leq 0 \quad \text{for } i = 1, \dots, k \\ h_i(x) = 0 \quad \text{for } i = 1, \dots, l \end{cases}$$

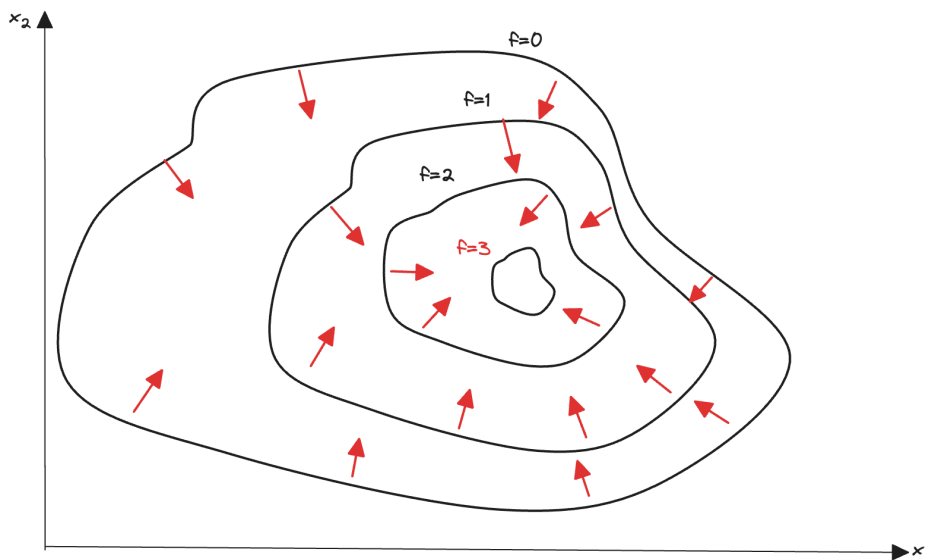
where $f()$ is the objective function, $g()$ are inequality constraints, and $h()$ are **equality constraints**. A powerful mathematical approach to solve this kind of problem is the **Lagrangian method**.

1.2.1 Equality constraints

We can consider first the simple problem with only one equality constraint:

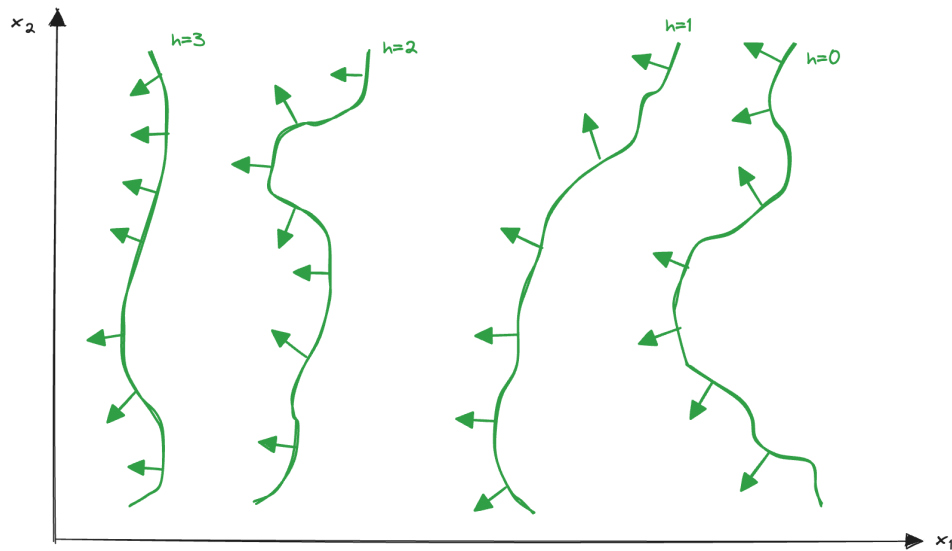
$$\begin{cases} \min_x f(x) \\ \text{s.t. } h(x) = 0 \end{cases}$$

The key idea is to transform the constrained problem into an equivalent one in which the constraints are **incorporated** into the objective function. To build some intuition for how this works, it is helpful to look at the geometry of the objective function. Consider a simple case where the variable is two-dimensional. In this setting, we can visualize the objective function by drawing curves along which the function takes the same value. These curves are called **level curves**, and they represent all points where the objective has constant value, as illustrated in the figure below:



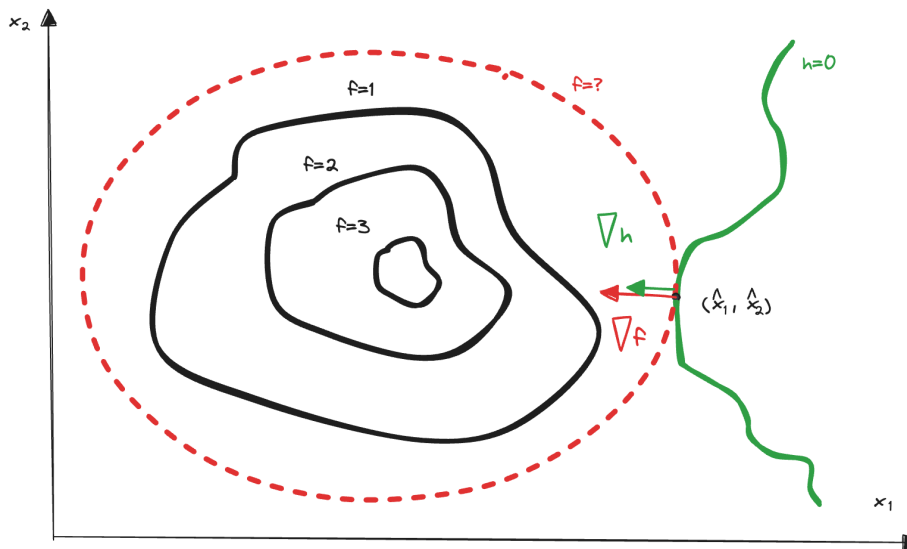
A useful geometric fact is that the gradient of the function is always perpendicular to these level curves. Intuitively, this is because the gradient points in the direction of steepest ascent, while moving along a level curve does not change the value of the function at all. We can think of this in terms of hiking: if you walk along a contour line on a map, your altitude stays constant, while

the steepest direction uphill is always perpendicular to our path. Now we can have a similar map for the equality constrain function:



However, we are not interested in the entire domain of the objective function. We are only allowed to consider points that satisfy the constraint. Geometrically, this means that instead of searching over the whole space, we are restricted to a curve (or surface) defined by the constraint. In the ideal case, the maximum of the function would lie exactly on the constraint. If that were true, the problem would be trivial: we could simply take that point as the solution. But in general, this does not happen. The unconstrained optimum typically lies outside the feasible region, so it is not allowed. As a result, we must look for **the best possible point along the constraint itself**. Geometrically, we can think of this as sliding along the level curves of the objective function and moving toward better values, until we reach the point where a level curve just touches the constraint. At this point, the objective cannot be improved any further without leaving the feasible set:

$$(\hat{x}_1, \hat{x}_2)$$



Why is this point special? Because any point outside this contour corresponds to a lower value of the objective, while any point inside the contour would give a higher value but would violate the constraint. Therefore, this point represents **the best possible trade-off**: it is the **optimal feasible solution**.

But how can we characterize this optimal point mathematically? The key insight comes from geometry. At the optimum, the level curve of the objective function and the constraint curve must be **tangent** to each other. That is, they touch at a single point without crossing. To understand why this must be the case, think about moving along the constraint. Now imagine that at the candidate optimal point the constraint is not tangent to the level curve. In that case, the constraint must cross the level curve. This has an important consequence: if we move slightly along the constraint in one direction, we will enter a region where the value of the objective function is higher. In other words, we could find a nearby point:

$$(\hat{x}_1 + \delta, \hat{x}_2 + \delta)$$

that satisfies the constraint and achieves a better objective value. But this contradicts the assumption that the previous point is optimal. Therefore, at the optimal point, such an improving direction cannot exist. The only way this can happen is if the constraint curve just touches the level curve of the objective, without crossing it. This is exactly the condition of tangency.

Now recall a fundamental geometric fact: the gradient of a function is always perpendicular to its level curves. Since the contour and the constraint are tangent at the optimum, **their normal directions must coincide**. This means that their gradients point in the same direction. In other words, at the optimal point, the gradients satisfy:

$$\nabla f(x) = \beta \nabla h(x)$$

where the scalar is needed because even if the two gradient are parallel, they might not have the same length, so there must be some factor allowing to transform one in the other. We can write this condition equivalently:

$$\nabla f(x) - \beta \nabla h(x) = 0$$

and we can define a new function called the **Lagrangian** that combines the objective function and the constraint function:

$$\mathcal{L}(x, \beta) = f(x) - \beta h(x)$$

and the condition to find the minimum of the objective function under the constraint becomes the condition to find the minimum of the Lagrangian:

$$\nabla \mathcal{L}(x, \beta) = 0$$

This condition is the core idea behind the method of Lagrange multipliers, and it provides a practical way to identify the optimal point.

As an example, we can consider the following optimization problem:

$$\begin{cases} \min_{x_1, x_2} f(x_1, x_2) = x_1^2 + x_2^2 \\ \text{s.t. } h(x_1, x_2) = x_1 + x_2 - 1 = 0 \end{cases}$$

We can write a Python implementation:

```
def f(x1, x2):  
    return x1**2 + x2**2  
  
def h(x1, x2):  
    return x1 + x2 - 1
```

The derivative of the two functions are:

$$\nabla f(x_1, x_2) = \begin{bmatrix} 2x_1 \\ 2x_2 \end{bmatrix}$$

$$\nabla h(x_1, x_2) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

```
def grad_f(x1, x2):  
    return 2*x1, 2*x2  
  
def grad_h(x1, x2):  
    return np.ones_like(x1), np.ones_like(x2)
```

We can prepare a grid of points in the feature space, and evaluate the objective function and its gradients at each point:

```
# Create the grid (feature space)
x1 = np.linspace(-1, 1, 15)
x2 = np.linspace(-1, 1, 15)
X1, X2 = np.meshgrid(x1, x2)

# Evaluate function and its gradient
F = f(X1, X2)
Gf1, Gf2 = grad_f(X1, X2)
```

This will allow us to visualize the level curves of the objective function and the direction of the gradients:

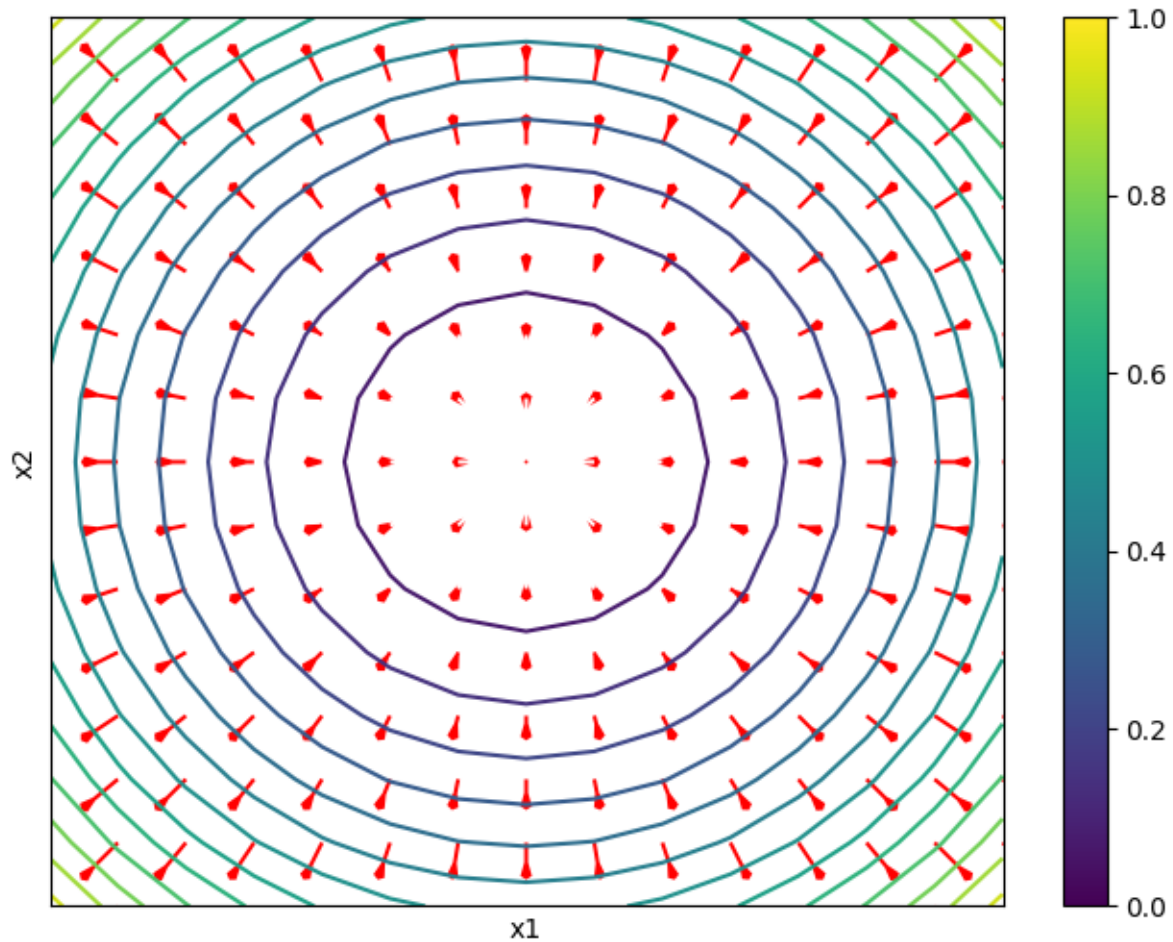
```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))

# Level curves (contours)
plt.contour(X1, X2, F, levels=15)

# Gradient field of the objective function
plt.quiver(X1, X2, Gf1, Gf2, color='r', headlength=1)

# Graph settings
plt.colorbar()
plt.xlabel('x1')
plt.ylabel('x2')
plt.xticks([])
plt.yticks([])
plt.show()
```



We can consider only points where the constraint is satisfied:

$$h(x_1, x_2) = 0$$

$$x_1 + x_2 - 1 = 0 \quad \rightarrow \quad x_2 = 1 - x_1$$

We can evaluate the constraint line and the gradients of the constraint function at those points:

```
# Constraint line
x1_line = np.linspace(0, 1, 25)
x2_line = 1 - x1_line

# Gradients evaluated along the constraint
Gf1, Gf2 = grad_f(x1_line, x2_line)
Gh1, Gh2 = grad_h(x1_line, x2_line)
```

In the graph, we can plot that line on top of the contour plot with also the gradients of the

constraint and objective functions, focusing the plot near the line and increasing the gradient vector length to highlight some details:

```
plt.figure(figsize=(8, 6))

# Filled contours of the objective function
contours = plt.contour(X1, X2, F, levels=15, cmap='viridis')

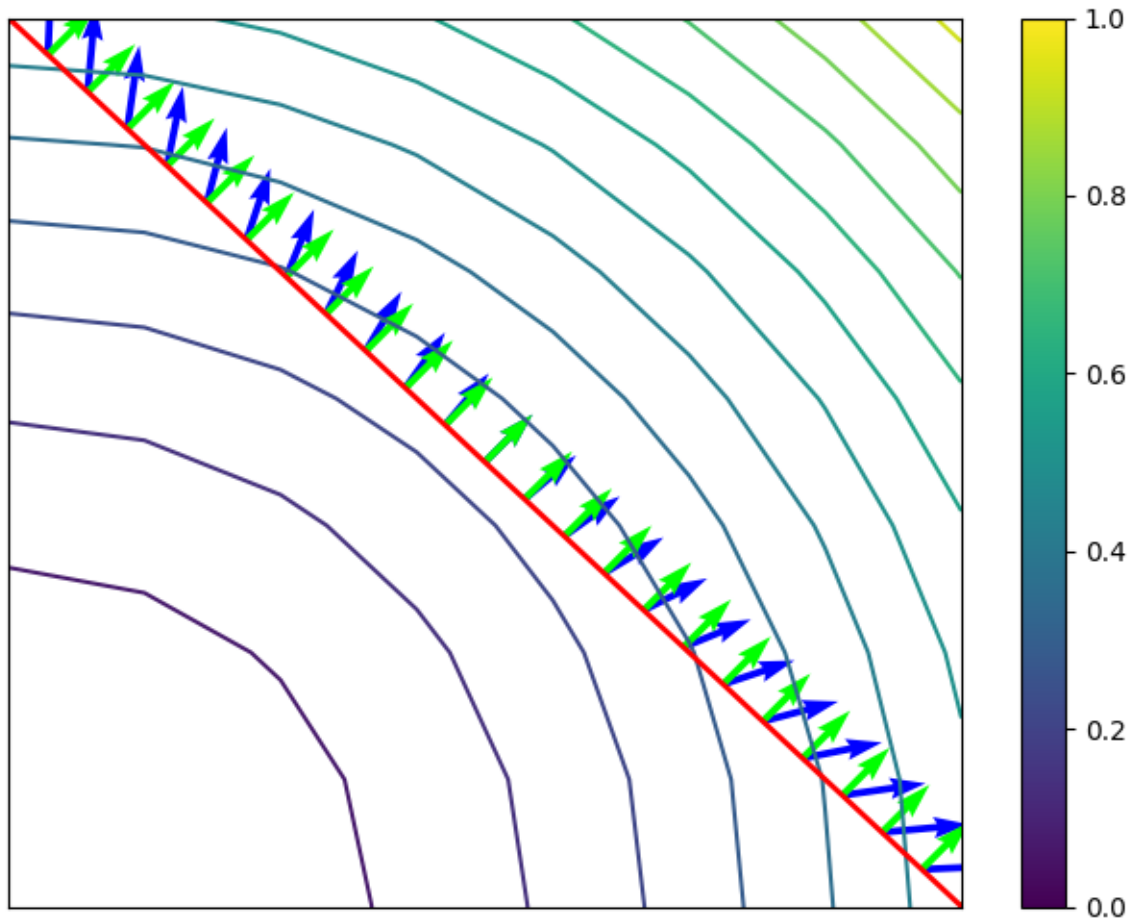
# Constraint line
plt.plot(x1_line, x2_line, color='red', linewidth=2)

# Gradient of the objective function along the constraint
plt.quiver(x1_line, x2_line, Gf1, Gf2, color='b', scale=20, headlength=5)

# Gradient of the constraint along the constraint
plt.quiver(x1_line, x2_line, Gh1, Gh2, color='lime', scale=20, headlength=5)

# Colorbar and axis settings
plt.colorbar()
plt.xlim([0, 1])
plt.ylim([0, 1])
plt.xticks([])
plt.yticks([])

plt.show()
```



We can now apply the Lagrangian method to compute the optimal point of the constrained problem. We combine the objective and the constraint into a single lagrangian function:

$$\mathcal{L}(x_1, x_2, \beta) = x_1^2 + x_2^2 + \beta(x_1 + x_2 - 1)$$

To find the optimal point, we look for stationary points of the Lagrangian. This means that we set all partial derivatives equal to zero:

$$\nabla \mathcal{L}(x_1, x_2, \beta) = \begin{bmatrix} 2x_1 + \beta \\ 2x_2 + \beta \\ x_1 + x_2 - 1 \end{bmatrix} = 0$$

Solving the system:

$$x_1 = \frac{1}{2}, \quad x_2 = \frac{1}{2}, \quad \beta = -1$$

This point corresponds to the optimal solution of the constrained problem. In this simple example, the optimality conditions lead to a system of equations that can be solved explicitly, yielding a closed-form solution. In general, however, constrained optimization problems

are not so tractable, and an **analytical solution is rarely available**. In those cases, we must rely on **numerical optimization methods**. A widely used tool for this purpose is the function `scipy.optimize.minimize`, which allows us to specify the objective function together with optional bounds and constraints. By applying different iterative algorithms, we can solve the problem numerically and verify that they all converge to the same solution. One of the most commonly used methods is **SLSQP**, which stands for **Sequential Least Squares Programming**. This is an iterative algorithm specifically designed for problems with smooth objectives and constraints, exactly like the ones we have been studying. The idea behind SLSQP is to approximate the original problem locally by a simpler one. At each iteration, the objective function is approximated by a quadratic model, while the constraints are linearized. This results in a quadratic program with linear constraints, which can be solved efficiently. The solution of this subproblem provides a direction along which the algorithm moves, and the process is repeated until convergence, that is, until the optimality conditions are satisfied. In practice, the algorithm requires only the objective function and the constraints. The necessary derivatives are approximated numerically, which makes the method easy to use. We can apply this method to our example problem, and we will find that it converges to the same optimal point that we derived analytically:

```
from scipy.optimize import minimize

result = minimize(
    lambda x: f(x[0], x[1]),          # objective: x1^2 + x2^2
    x0=[0, 0],                        # starting point
    method='SLSQP',
    constraints={'type': 'eq', 'fun': lambda x: h(x[0], x[1])} # h(x1, x2) = 0
)

x1_opt, x2_opt = result.x
print(f"Optimal point:      x1 = {x1_opt:.4f}, x2 = {x2_opt:.4f}")
```

Optimal point: x1 = 0.5000, x2 = 0.5000

1.2.2 Multiple equality and inequality constraints

The method of Lagrange multipliers extends naturally to problems with **multiple equality constraints**, and can be further generalized to handle **inequality constraints**. A general constrained optimization problem can be written as:

$$\begin{cases} \min_x f(x) \\ \text{s.t. } g_i(x) \leq 0 \quad \text{for } i = 1, \dots, k \\ h_i(x) = 0 \quad \text{for } i = 1, \dots, l \end{cases}$$

The functions $h()$ represent equality constraints and functions $g()$ represent inequality constraints. Each constraint restricts the set of feasible solutions: equality constraints define surfaces (or curves) on which the solution must lie, while inequality constraints define regions of the space. The feasible set is therefore the **intersection of all these regions and surfaces**, and the optimization problem consists of finding the best point of the objective function within this restricted set. To handle all constraints simultaneously, we introduce **one multiplier for each constraint** and define the **generalized Lagrangian**:

$$\mathcal{L}(x, \alpha, \beta) = f(x) + \sum_{i=1}^k \alpha_i g_i(x) + \sum_{j=1}^l \beta_j h_j(x)$$

This expression combines the objective function with all the constraints into a single quantity. Each constraint contributes a term weighted by its corresponding multiplier, which can be interpreted as controlling how strongly that constraint influences the solution. A key difference between equality and inequality constraints lies in the role of their multipliers. For inequality constraints, we must impose:

$$\alpha_i \geq 0$$

Each inequality term acts as a penalty when the constraint is violated, in other words when the value of $g()$ is positive. In that condition, we want the Lagrangian to increase so that such points become less favorable. Requiring positive parameters guarantees exactly this behavior: violations always lead to an increase in the objective, never to a decrease. In this way, the Lagrangian naturally discourages infeasible solutions, guiding the optimization toward points that satisfy all the constraints. As in the previous simpler case, the role of the Lagrangian is to incorporate all constraints into a single function. **Instead of explicitly restricting the domain, we modify the objective so that violating a constraint is penalized through the multipliers.** In this way, the constrained optimization problem can be analyzed using tools similar to unconstrained optimization, while still respecting all the imposed restrictions.

To make this idea precise, we want to build a function that **automatically excludes infeasible points** by assigning them a very large cost. In other words, we want a function that behaves like the original objective on feasible points, but "blows up" whenever a constraint is violated. This is called the **primal function**:

$$\theta_P(x) = \max_{\alpha, \beta: \alpha_i \geq 0} \mathcal{L}(x, \alpha, \beta)$$

For a given x , we are allowed to choose the multipliers in the most "adversarial" way, that is, in the way that makes the Lagrangian as large as possible. If the point violates any constraint,

then the Lagrangian can be made arbitrarily large. For example:

- if $g_i(x) > 0$, we can send $\alpha_i \rightarrow +\infty$
- if $h_j(x) \neq 0$, we can send $\beta_j \rightarrow \pm\infty$

In both cases, the Lagrangian "blows up" and we obtain:

$$\theta_P(x) = \infty$$

On the other hand, if the point satisfies all the constraints, then all the constraint terms vanish or are non-positive, and the maximum is achieved without "penalties", giving:

$$\theta_P(x) = f(x)$$

So the primal function behaves like a penalized version of the objective:

$$\theta_P(x) = \begin{cases} f(x) & \text{if } x \text{ satisfies the constraints} \\ \infty & \text{otherwise} \end{cases}$$

This is a crucial observation. The primal function behaves exactly like the original objective function on feasible points, but assigns an infinite cost to any point that violates the constraints. As a result, minimizing it automatically restricts the search to feasible solutions, since any infeasible point is effectively excluded. In this way, the original constrained problem can be rewritten as an equivalent unconstrained one:

$$p^* = \min_x \theta_P(x) = \min_x \max_{\alpha, \beta: \alpha_i \geq 0} \mathcal{L}(x, \alpha, \beta)$$

which is called the **primal optimal value**.

1.2.3 Dual formulation

Although the primal formulation already gives us a way to solve the problem, it is often useful to look at it from a **different perspective**. In particular, by exchanging the roles of the variables and the multipliers, we obtain an alternative formulation that can reveal additional structure and, in many cases, lead to a more tractable optimization problem. Instead of fixing x and maximizing over the multipliers, we can **fix the multipliers and minimize over x** . This leads to the function:

$$\theta_D(\alpha, \beta) = \min_x \mathcal{L}(x, \alpha, \beta)$$

which is called the **dual function**. To understand its meaning, consider that each choice of multipliers defines a different "approximation" of the original problem, and the dual function tells us how good that approximation can be. Different multipliers lead to different values, so it is natural to look for the choice that gives the best possible result. This leads to the dual optimization problem:

$$d^* = \max_{\alpha, \beta: \alpha_i \geq 0} \theta_D(\alpha, \beta) = \max_{\alpha, \beta: \alpha_i \geq 0} \min_x \mathcal{L}(x, \alpha, \beta)$$

which is called the **dual optimal value**. Intuitively, the dual problem searches for the choice of multipliers that makes the Lagrangian as "tight" as possible, providing the best achievable value under this penalized formulation.

The primal and dual problems are deeply connected, and this connection comes from a simple but powerful observation: they differ only in the order in which we take the minimum over x and the maximum over the multipliers. These two operations **do not generally commute**. The primal allows the multipliers to react after seeing the choice of x , while the dual forces the multipliers to be chosen before x is selected. This makes the dual inherently more "conservative". As a consequence, the dual can never achieve a value larger than the primal.

$$d^* = \max_{\alpha, \beta: \alpha_i \geq 0} \min_x \mathcal{L}(x, \alpha, \beta) \leq \min_x \max_{\alpha, \beta: \alpha_i \geq 0} \mathcal{L}(x, \alpha, \beta) = p^*$$

This result is known as **weak duality**, and it can be proven as follows. The minimum value over x is always less than or equal to the value at any particular point x :

$$\min_{x'} \mathcal{L}(x', \alpha, \beta) \leq \mathcal{L}(x, \alpha, \beta)$$

Now, if we take the maximum over all multipliers on both sides, we obtain

$$\max_{\alpha, \beta: \alpha_i \geq 0} \min_{x'} \mathcal{L}(x', \alpha, \beta) \leq \max_{\alpha, \beta: \alpha_i \geq 0} \mathcal{L}(x, \alpha, \beta)$$

This holds for every x , therefore, we can take the minimum over x on the right-hand side without violating the inequality:

$$\max_{\alpha, \beta: \alpha_i \geq 0} \min_x \mathcal{L}(x, \alpha, \beta) \leq \min_x \max_{\alpha, \beta: \alpha_i \geq 0} \mathcal{L}(x, \alpha, \beta)$$

That show the weak duality result. However, under certain conditions, the dual problem can achieve the same optimal value as the primal problem, leading to what is called **strong duality**:

$$d^* = p^*$$

This situation is guaranteed under a set of conditions known as **Slater's condition**. In essence, these require that the problem has a well-behaved structure: the objective function and the inequality constraints are **convex**, the equality constraints are **linear**, and there **exists at least one point that strictly satisfies all inequality constraints**. The intuition behind these assumptions is geometric. They ensure that the feasible region is not "flattened" against its boundary, but has some interior. This prevents pathological situations in which the optimal solution lies on a degenerate set where the primal and dual formulations fail to match, creating a gap between them.

When Slater's condition holds, strong duality is guaranteed. But there is an even more powerful consequence: the optimal primal and dual variables can be completely characterized by

a system of conditions known as the **Karush-Kuhn-Tucker (KKT) conditions**. Under Slater's condition, these are both necessary and sufficient for optimality (meaning that finding a point that satisfies them is equivalent to solving the original problem). The KKT conditions require four things to hold simultaneously at the optimum:

Stationarity: at the optimal point there is no direction that simultaneously improves the objective and remains feasible. This means the gradient of the Lagrangian must vanish: any improvement in the objective is exactly counterbalanced by the cost imposed by the constraints through the multipliers.

$$\nabla_x \mathcal{L}(x^*, \alpha^*, \beta^*) = 0$$

Primal feasibility: the solution must lie in the feasible region by definition. This simply restates the original constraints and ensures that the optimal point we find is actually admissible.

$$g_i(x^*) \leq 0, \quad h_j(x^*) = 0$$

Dual feasibility: this is the same requirement imposed when building the generalized Lagrangian. The multipliers of the inequality constraints must be non-negative so that violations increase the Lagrangian rather than decrease it, correctly penalizing infeasible points.

$$\alpha_i^* \geq 0$$

Complementary slackness: Recall that each inequality constraint contributes a term to the Lagrangian. At the optimum this term must vanish for every constraint. There are exactly two ways this can happen. If the constraint is **inactive**

$$g_i(x^*) < 0$$

the point lies strictly inside the feasible region. The constraint is not pressing on the solution and should have no influence on it. Setting

$$\alpha_i^* = 0$$

removes its contribution from the Lagrangian entirely. If the constraint is **active**

$$g_i(x^*) = 0$$

the point lies exactly on the boundary of the feasible region. The constraint is binding and may carry a nonzero multiplier

$$\alpha_i^* \geq 0$$

that measures how strongly it pushes against the optimum. In both cases we have

$$\alpha_i^* g_i(x^*) = 0$$

This is not an assumption but a consequence of optimality: at the optimum, a constraint either matters or it does not. There is no middle ground. We will see that this condition is precisely

what identifies the support vectors in the SVM problem.

This is extremely valuable in practice: many optimization problems (including SVMs) become **significantly easier to solve in their dual form**.

1.3 Optimal margin classifier

We now apply the machinery of constrained optimization to classification, leading to the optimal margin classifier, which selects the separating hyperplane that maximizes the geometric separation between the two classes.

1.3.1 Separable case

First of all, we consider the case where we have two classes and the data is **linearly separable**. In this setting, we make the assumption that there **exists a hyperplane that perfectly separates the two classes**, in other words, there is an hyperplane:

$$w^T x + b = 0$$

that places all points of one class on one side, and all points of the other class on the other side. Translating into math, for a specific point we have two possibilities:

- if $w^T x^{(i)} + b > 0 \rightarrow$ one side
- if $w^T x^{(i)} + b < 0 \rightarrow$ the other side

Now we use the labels:

$$y^{(i)} \in -1, +1$$

to encode both conditions in a single compact condition:

$$y^{(i)}(w^T x^{(i)} + b) > 0$$

We apply the Lagrangian framework to the SVM optimization problem:

$$\begin{cases} \min_{w,b} \frac{1}{2} w^T w \\ \text{s.t. } y^{(i)}(w^T x^{(i)} + b) \geq 1 \quad \text{for } i = 1, \dots, m \end{cases}$$

To bring this problem into the standard form for inequality constraints, we rewrite each constraint as

$$g_i(w, b) = 1 - y^{(i)}(w^T x^{(i)} + b) \leq 0 \quad \text{for } i = 1, \dots, m$$

so that **each training example contributes one constraint**, to enforce that every point lies not only on the correct side of the hyperplane, but also **outside the margin boundaries**. We now write the Lagrangian by introducing one multiplier for each constraint:

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} w^T w + \sum_{i=1}^m \alpha_i (1 - y^{(i)} (w^T x^{(i)} + b))$$

This optimization problem has a particularly favorable structure. The objective function is **convex (being quadratic in w)**, and the **constraints are affine**, since they are linear in the parameters. Problems with this structure fall into the class of **convex optimization problems**, for which strong theoretical guarantees are available. Finally, since we assumed that the data is linearly separable, there exist parameters such that:

$$y^{(i)} (w_0^T x^{(i)} + b_0) > 0 \quad \text{for all } i$$

define

$$\gamma = \min_i y^{(i)} (w_0^T x^{(i)} + b_0)$$

By construction, all terms are strictly positive, so

$$\gamma > 0$$

Now consider a rescaled version of these parameters:

$$w = c w_0, \quad b = c b_0$$

Then

$$y^{(i)} (w^T x^{(i)} + b) = c y^{(i)} (w_0^T x^{(i)} + b_0) \geq c \gamma$$

By choosing

$$c > \frac{1}{\gamma}$$

we obtain

$$y^{(i)} (w^T x^{(i)} + b) > 1 \quad \text{for all } i$$

This shows explicitly that there exist strictly feasible points, and therefore the **Slater's condition is satisfied**. As a consequence, **strong duality holds**, which means that the optimal value of the dual problem coincides exactly with that of the primal problem. In practical terms, this is extremely important: it guarantees that we can solve the dual problem instead of the primal one without losing any information, while often gaining a simpler and more structured formulation. Let's consider the dual form of the problem:

$$\max_{\alpha} \min_{w, b} \mathcal{L}(w, b, \alpha) = \max_{\alpha} \min_{w, b} \frac{1}{2} w^T w + \sum_{i=1}^m \alpha_i (1 - y^{(i)} (w^T x^{(i)} + b))$$

Since Slater's condition is satisfied, the KKT conditions are necessary and sufficient. Applying the **stationarity condition**, we set the gradient of the Lagrangian with respect to the primal variables to zero:

$$\nabla_w \mathcal{L}(w, b, \alpha) = w - \sum_{i=1}^m \alpha_i y^{(i)} x^{(i)} = 0$$

$$\nabla_b \mathcal{L}(w, b, \alpha) = - \sum_{i=1}^m \alpha_i y^{(i)} = 0$$

These conditions immediately give two important relations. From the derivative with respect to w , we obtain an explicit expression for the normal vector:

$$w = \sum_{i=1}^m \alpha_i y^{(i)} x^{(i)}$$

From the derivative with respect to b , we obtain the constraint

$$\sum_{i=1}^m \alpha_i y^{(i)} = 0$$

We now plug the expression for w back into the Lagrangian. The goal is to eliminate the primal variables and rewrite the problem entirely in terms of the multipliers. Starting from the original expression:

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} w^T w + \sum_{i=1}^m \alpha_i (1 - y^{(i)} (w^T x^{(i)} + b))$$

Consider the first term:

$$\frac{1}{2} w^T w = \frac{1}{2} \left(\sum_{i=1}^m \alpha_i y^{(i)} x^{(i)} \right)^T \left(\sum_{j=1}^m \alpha_j y^{(j)} x^{(j)} \right) = \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)}$$

Then consider the second term:

$$\begin{aligned} \sum_{i=1}^m \alpha_i (1 - y^{(i)} (w^T x^{(i)} + b)) &= \sum_{i=1}^m \alpha_i - \sum_{i=1}^m \alpha_i y^{(i)} w^T x^{(i)} - b \sum_{i=1}^m \alpha_i y^{(i)} = \sum_{i=1}^m \alpha_i - \sum_{i=1}^m \alpha_i y^{(i)} \left(\sum_{j=1}^m \alpha_j y^{(j)} x^{(j)} \right)^T x^{(i)} - \\ b \sum_{i=1}^m \alpha_i y^{(i)} &= \sum_{i=1}^m \alpha_i - \sum_{i,j=1}^m \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)} - b \sum_{i=1}^m \alpha_i y^{(i)} \end{aligned}$$

The previously obtained constraint allows us to eliminate the last term:

$$= \sum_{i=1}^m \alpha_i - \sum_{i,j=1}^m \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)}$$

Putting everything together, we obtain a formulation that depends only on the multipliers:

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)} + \sum_{i=1}^m \alpha_i - \sum_{i,j=1}^m \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)} = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)}$$

The dual optimization problem can now be written as:

$$\begin{cases} \max_{\alpha: \alpha_i \geq 0} \left(\sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)} \right) \\ \text{s.t. } \sum_{i=1}^m \alpha_i y^{(i)} = 0 \end{cases}$$

The dual problem falls into the same category of constrained optimization problems we encountered earlier. Unlike the simple example, here **a closed-form solution does not exist**: with m training points we have m multipliers to optimize simultaneously and the KKT stationarity conditions do not reduce to a solvable linear system. We therefore apply the same numerical approach seen before.

```
def dual_objective_hard(alpha):
    return alpha.sum() - 0.5 * (alpha * y) @ X @ X.T @ (alpha * y)
```

Applying this to the Iris dataset:

```
result = minimize(
    lambda alpha: -dual_objective_hard(alpha),      # maximize = minimize the negative
    x0=np.zeros(X.shape[0]),                        # one alpha per training point
    method='SLSQP',
    bounds=[(0, None)] * X.shape[0],              # alpha_i ≥ 0
    constraints={'type': 'eq', 'fun': lambda a: a @ y}, # sum(alpha_i * y_i) = 0
    options={'ftol': 1e-9, 'maxiter': 1000}
)

alpha = result.x
```

An important consequence of complementary slackness is that **most multipliers are exactly zero at the optimum**. Only the points that lie on the margin boundary (the support vectors) have a positive multiplier. **All other training points do not contribute to the solution at all**. This means that the optimal hyperplane depends only on the support vectors, making the solution **sparse**: the decision boundary is fully determined by a small subset of the training data. We can verify this directly:

```
n_support = (alpha > 1e-5).sum()
print(f"Support vectors: {n_support} out of {len(alpha)} training points")
for i, a in enumerate(alpha):
    if a > 1e-5:
        print(f"  point {i}: alpha = {a:.4f}, y = {int(y[i])}, x = {X[i]}")
```

```
Support vectors: 2 out of 100 training points
  point 44: alpha = 1.1765, y = -1, x = [1.9 0.4]
  point 98: alpha = 1.1765, y = 1, x = [3. 1.1]
```

Once the optimal multipliers are found, we can reconstruct the normal vector directly from them:

$$w = \sum_{i=1}^m \alpha_i y^{(i)} x^{(i)}$$

```
w = (alpha * y) @ X
```

However, the bias term does not appear explicitly in the dual solution and must be recovered using an additional step. The key idea is to use the fact that, at the optimum, the training points with the multipliers non-zero (**support vectors**) lie exactly on the margin boundaries (**complementary slackness KKT condition**). For each of these points, the corresponding constraint is active, which means:

$$y^{(i)}(w^T x^{(i)} + b) = 1$$

This equation allows us to solve for b. Rearranging, we obtain

$$w^T x^{(i)} + b = y^{(i)} \Rightarrow b = y^{(i)} - w^T x^{(i)}$$

In practice, any support vector can be used to compute b. However, for better numerical stability, it is common to average this quantity over all support vectors:

$$b = \frac{1}{|S|} \sum_{i \in S} (y^{(i)} - w^T x^{(i)})$$

where S is the set of indices corresponding to support vectors. This procedure ensures a more robust estimate of the bias.

```
sv_mask = alpha > 1e-5
sv_X, sv_y = X[sv_mask], y[sv_mask]

b = np.mean(sv_y - sv_X @ w)
```

Once the parameters of the hyperplane are known, we can visualize the solution geometrically. In a two-dimensional feature space, the decision boundary

$$w^T x + b = 0$$

is a line. Solving for the second feature:

$$x_2 = -\frac{w_1}{w_2} x_1 - \frac{b}{w_2}$$

The two margin boundaries lie at

$$w^T x + b = +1$$

and

$$w^T x + b = -1$$

which are parallel lines shifted by

$$\frac{1}{w_2}$$

from the decision boundary. The total width of the margin is therefore

$$\frac{2}{|w|}$$

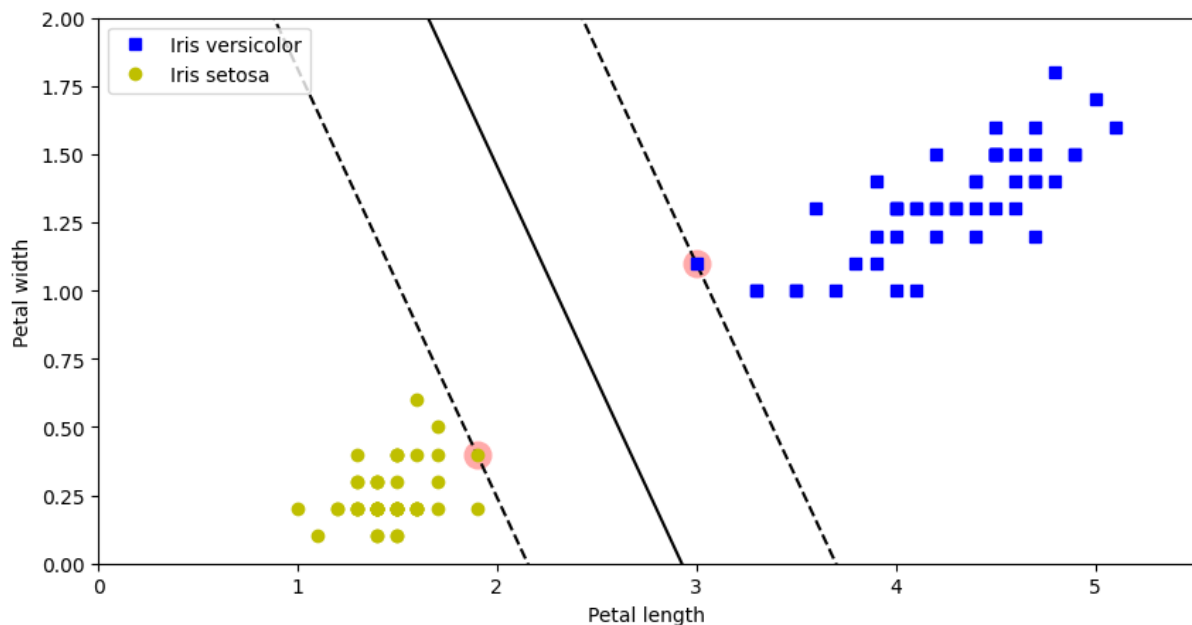
as derived earlier. The support vectors lie exactly on these boundaries and are the only training examples that determine the position of the hyperplane.

```
def plot_svm_decision_boundary(w, b, support_vectors, xmin, xmax, xlabel, ylabel):
    x = np.linspace(xmin, xmax, 200)
    decision_boundary = -w[0]/w[1] * x - b/w[1]
    margin = 1/w[1]

    fig, ax = plt.subplots(figsize=(10, 5))
    ax.scatter(support_vectors[:, 0], support_vectors[:, 1], s=180,
        facecolors='#FFAAAA')
    ax.plot(x, decision_boundary, "k-")
    ax.plot(x, decision_boundary + margin, "k--")
    ax.plot(x, decision_boundary - margin, "k--")
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
```

Applying this to the Iris dataset, the support vectors (highlighted) lie exactly on the margin boundaries, and the decision boundary is positioned to maximize the gap between the two classes:

```
plot_svm_decision_boundary(w, b, sv_X, 0, 5.5, "Petal length", "Petal width")
plt.plot(X[:, 0][y==1], X[:, 1][y==1], "bs", label="Iris versicolor")
plt.plot(X[:, 0][y==0], X[:, 1][y==0], "yo", label="Iris setosa")
plt.legend(loc="upper left")
plt.axis([0, 5.5, 0, 2])
plt.show()
```



Finally, the optimal primal parameters can be used to make predictions for new data points. The prediction for a new point is:

$$h_{w,b}(x) = \text{sign}(w^T x + b)$$

Notice that adding more training instances “off the street” will not affect the decision boundary at all: it is fully determined (or supported) by the support vectors.

1.3.2 Non-separable case

The optimal margin classifier we have just derived assumes that the data is linearly separable: it looks for a hyperplane that places every training point on the correct side of the margin, with no exceptions. This is called **hard margin classification**, because the margin constraints are enforced strictly:

$$y^{(i)}(w^T x^{(i)} + b) \geq 1 \quad \text{for all } i$$

In practice, this assumption is **fragile**. Real datasets are **rarely perfectly separable**, and even when they are, hard margin classifiers are sensitive to **outliers**: a single misplaced point can make the problem infeasible, or force the margin to collapse to avoid it. For example, adding one outlier to the Iris dataset can make the two classes no longer linearly separable, leaving no valid hard margin solution:

```

X_outlier = np.array([[3.4, 1.3]])
y_outlier = np.array([-1])

Xo = np.concatenate([X, X_outlier], axis=0)
yo = np.concatenate([y, y_outlier], axis=0)

fig, ax = plt.subplots(figsize=(10, 5))

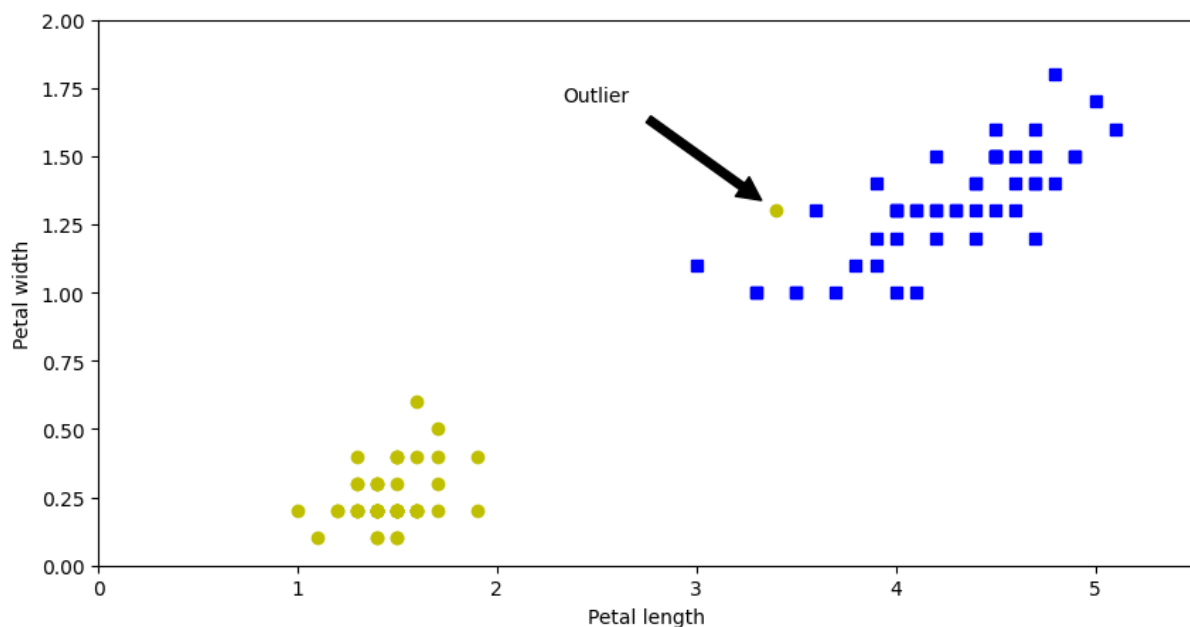
ax.plot(Xo[:, 0][yo==1], Xo[:, 1][yo==1], "bs")
ax.plot(Xo[:, 0][yo==-1], Xo[:, 1][yo==-1], "yo")

ax.set_xlabel("Petal length")
ax.set_ylabel("Petal width")

ax.annotate("Outlier",
            xy=(X_outlier[0][0], X_outlier[0][1]),
            xytext=(2.5, 1.7),
            ha="center",
            arrowprops=dict(facecolor='black', shrink=0.1))
ax.axis([0, 5.5, 0, 2])

plt.show()

```



Even when a hard margin solution exists, it is not necessarily a good one. Because the decision boundary is determined entirely by the support vectors, a single outlier close to or inside the

margin can shift it dramatically. The resulting hyperplane may fit the training data perfectly but generalize poorly to new points. For example, moving the outlier slightly inside the margin region produces a very different decision boundary:

```
X_outlier = np.array([[3.2, 0.8]])
y_outlier = np.array([-1])

Xo = np.concatenate([X, X_outlier], axis=0)
yo = np.concatenate([y, y_outlier], axis=0)

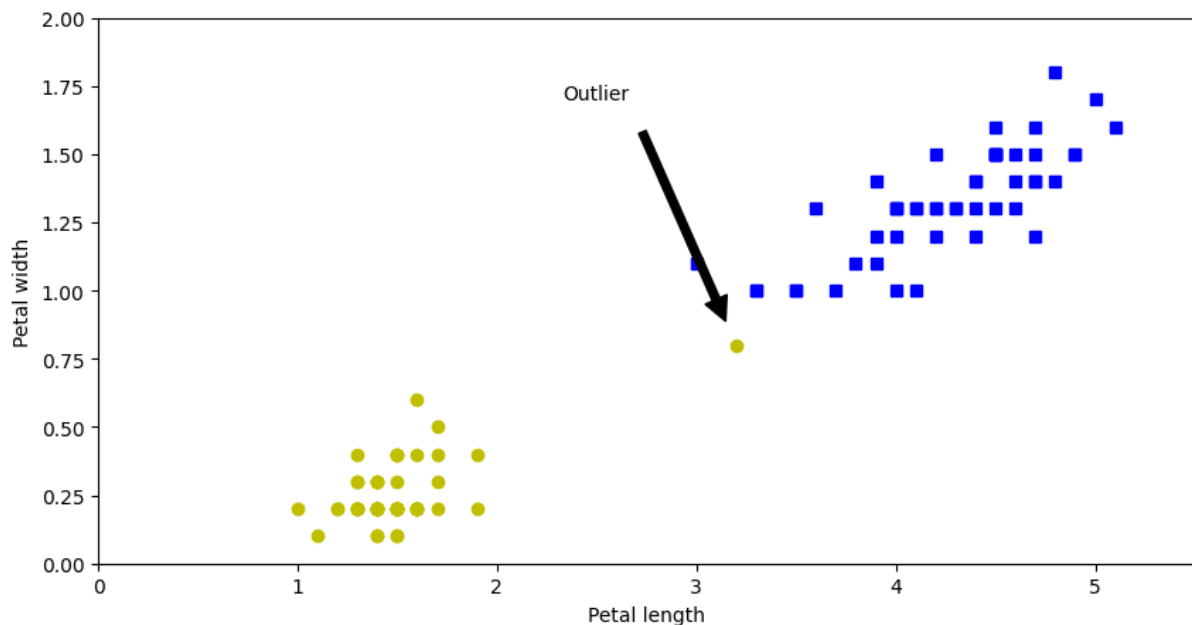
fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(Xo[:, 0][yo==1], Xo[:, 1][yo==1], "bs")
ax.plot(Xo[:, 0][yo==-1], Xo[:, 1][yo==-1], "yo")

ax.set_xlabel("Petal length")
ax.set_ylabel("Petal width")

ax.annotate("Outlier",
            xy=(X_outlier[0][0], X_outlier[0][1]),
            xytext=(2.5, 1.7),
            ha="center",
            arrowprops=dict(facecolor='black', shrink=0.1))
ax.axis([0, 5.5, 0, 2])

plt.show()
```



We can retrain the hard margin classifier on the dataset with the outlier using the same dual approach, and observe how the decision boundary changes:

```
# Prepare the dual problem with the outlier
def dual_objective_hard(alpha):
    return alpha.sum() - 0.5 * (alpha * yo) @ Xo @ Xo.T @ (alpha * yo)

# Solve the dual problem with the outlier
result_o = minimize(
    lambda alpha: -dual_objective_hard(alpha),      # maximize = minimize the negative
    x0=np.zeros(Xo.shape[0]),                      # one alpha per training point
    method='SLSQP',
    bounds=[(0, None)] * Xo.shape[0],              # alpha_i ≥ 0
    constraints={'type': 'eq', 'fun': lambda a: a @ yo}, # sum(alpha_i * y_i) = 0
    options={'ftol': 1e-9, 'maxiter': 1000}
)

# Extract the optimal alphas
alpha_o = result_o.x

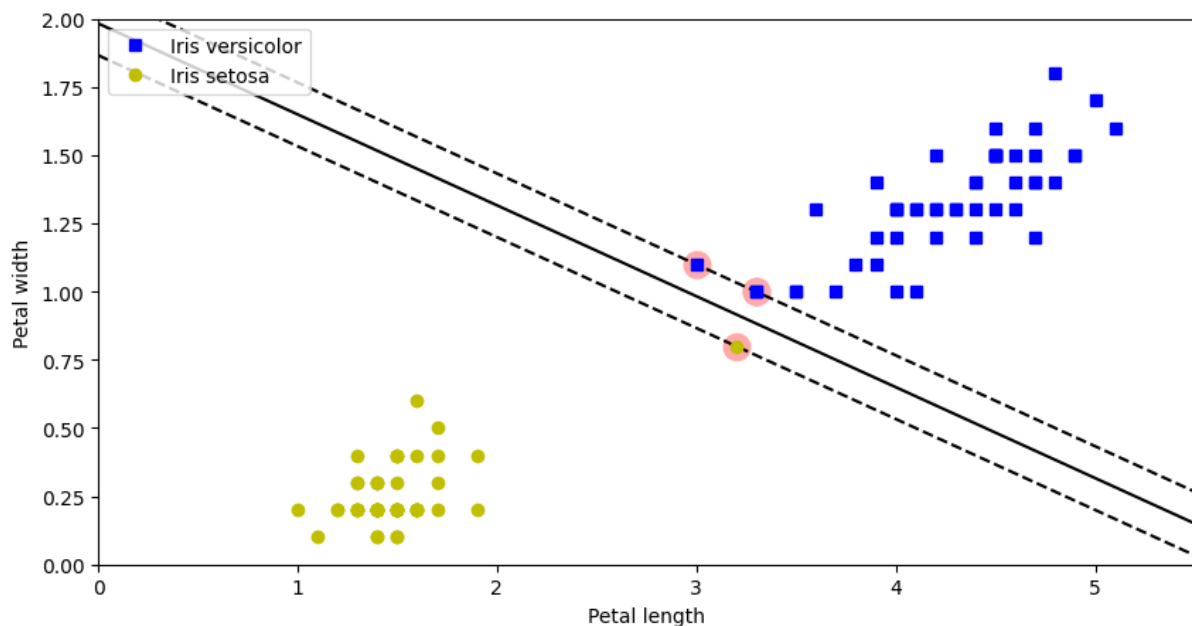
# Compute the new weight vector and bias term
wo = (alpha_o * yo) @ Xo
sv_mask_o = alpha_o > 1e-5
sv_Xo, sv_yo = Xo[sv_mask_o], yo[sv_mask_o]
bo = np.mean(sv_yo - sv_Xo @ wo)
```

```

plot_svm_decision_boundary(wo, bo, sv_Xo, 0, 5.5, "Petal length", "Petal width")

plt.plot(Xo[:, 0][yo==1], Xo[:, 1][yo==1], "bs", label="Iris versicolor")
plt.plot(Xo[:, 0][yo==-1], Xo[:, 1][yo==-1], "yo", label="Iris setosa")
plt.legend(loc="upper left")
plt.axis([0, 5.5, 0, 2])
plt.show()

```



The decision boundary has rotated significantly compared to the original one, pulled toward the outlier which has become a support vector. The margin is now much narrower and the hyperplane is **clearly less robust**, it would likely misclassify new versicolor points in the region the boundary has abandoned. The fundamental problem is that the hard margin formulation gives equal weight to every training point: a single outlier can completely determine the solution. We need a more **flexible approach** that **allows some points to violate the margin constraints**, while still penalizing such violations. The idea is to introduce one **slack variable**:

$$\zeta_i \geq 0$$

for each training point, measuring by how much that point is allowed to violate the margin:

- $\zeta_i = 0$: the point lies on the correct side of the margin, no violation
- $0 < \zeta_i < 1$: the point is inside the margin but on the correct side of the decision boundary
- $\zeta_i \geq 1$: the point crosses the decision boundary and is misclassified

The soft margin constraint then becomes:

$$y^{(i)}(w^T x^{(i)} + b) \geq 1 - \zeta_i \quad \text{for } i = 1, \dots, m$$

which reduces to the hard margin constraint when all slack variables are zero. The optimization problem now has **two competing objectives: maximize the margin and limit the total amount of violation**. A hyperparameter C controls the trade-off between the two:

$$\begin{cases} \min_{w,b,\zeta} \frac{1}{2} w^T w + C \sum_{i=1}^m \zeta_i \\ \text{s.t. } y^{(i)}(w^T x^{(i)} + b) \geq 1 - \zeta_i \quad \text{for } i = 1, \dots, m \end{cases}$$

A **large C** penalizes violations heavily, pushing the solution toward a narrow margin with few errors, closer to hard margin behavior, with a higher risk of overfitting. A **small C** tolerates more violations in exchange for a wider margin, which can improve generalization but may underfit if taken too far.

As in the hard margin case, this problem can be solved via the Lagrangian method by introducing one multiplier for each margin constraint and one multiplier for each non-negativity constraint on the slack variables:

$$\mathcal{L}(w, b, \zeta, \alpha, \mu) = \frac{1}{2} w^T w + C \sum_{i=1}^m \zeta_i - \sum_{i=1}^m \alpha_i [y^{(i)}(w^T x^{(i)} + b) - 1 + \zeta_i] - \sum_{i=1}^m \mu_i \zeta_i$$

The dual formulation follows the same steps as before. The key difference introduced by the slack variables becomes visible when we apply the stationarity condition with respect to ζ_i :

$$\frac{\partial \mathcal{L}}{\partial \zeta_i} = C - \alpha_i - \mu_i = 0 \quad \Rightarrow \quad \alpha_i + \mu_i = C$$

Since both multipliers must satisfy dual feasibility, this equation immediately constrains the multipliers:

$$\mu_i = C - \alpha_i \geq 0 \quad \Rightarrow \quad \alpha_i \leq C$$

Combined with the dual feasibility, we obtain the **box constraint**:

$$0 \leq \alpha_i \leq C$$

This is the only change in the dual problem compared to the hard margin case: the multipliers now have an upper bound in addition to the lower bound. Following the same steps as before, we can eliminate the primal variables and rewrite the dual problem entirely in terms of the multipliers:

$$\begin{cases} \max_{0 \leq \alpha_i \leq C} \left(\sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)} \right) \\ \text{s.t. } \sum_{i=1}^m \alpha_i y^{(i)} = 0 \end{cases}$$

The objective is identical to the hard margin dual, only the feasible region for the multipliers has changed. The upper bound C **limits how much any single training point can influence the solution**. This is the Python implementation:

```
C = 1.0

def dual_objective_soft(alpha):
    return alpha.sum() - 0.5 * (alpha * y) @ X @ X.T @ (alpha * y)
```

To test the soft margin classifier on a more realistic example, we move away from the artificially constructed outlier and consider a genuinely non-separable problem. We use the Iris dataset, this time distinguishing Iris virginica from the other two classes. Unlike the setosa vs versicolor case, these classes overlap in the petal length and petal width feature space and **cannot be perfectly separated** by any hyperplane.

```
X = iris["data"][:, (2, 3)]
y = np.where(iris["target"] == 2, 1, -1).astype(float) # virginica=+1, rest=-1
```

Solving the dual problem with the same numerical approach, mapping the box constraint directly onto the bounds:

```
result = minimize(
    lambda alpha: -dual_objective_soft(alpha),
    x0=np.zeros(X.shape[0]),
    method='SLSQP',
    bounds=[(0, C)] * X.shape[0], # alpha_i ≤ C only change
    constraints={'type': 'eq', 'fun': lambda a: a @ y},
    options={'ftol': 1e-9, 'maxiter': 1000}
)

alpha = result.x
```

The recovery of w and b from the optimal multipliers follows exactly the same steps as in the hard margin case, since the dual variables have the same interpretation:

```
w = (alpha * y) @ X

sv_mask = alpha > 1e-5
sv_X, sv_y = X[sv_mask], y[sv_mask]

b = np.mean(sv_y - sv_X @ w)
```

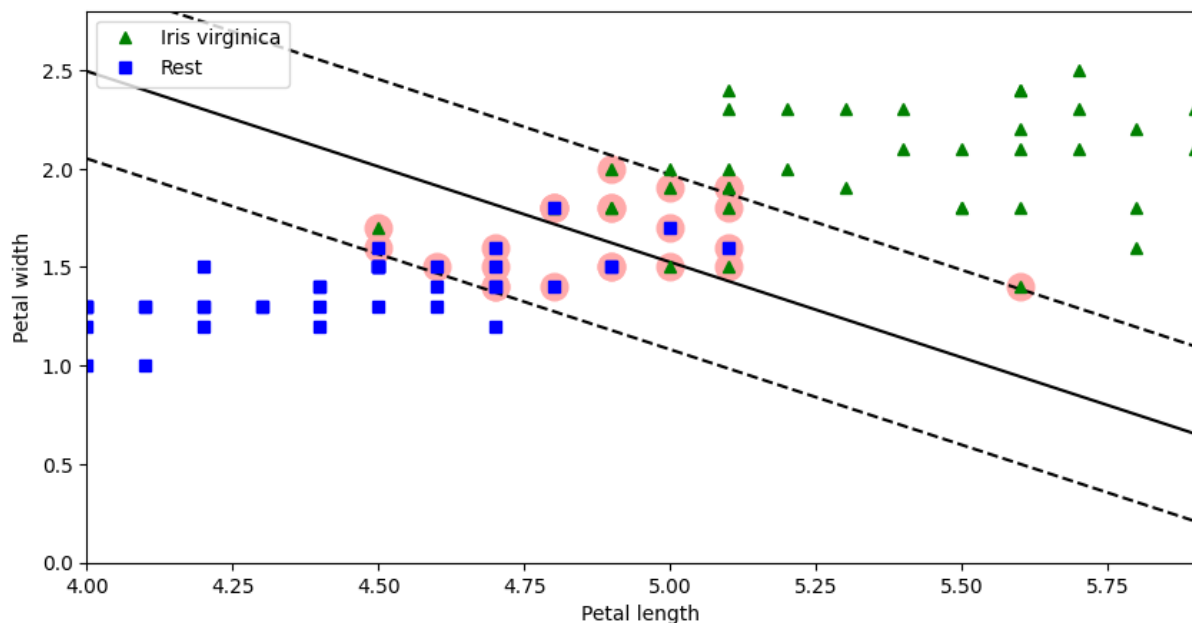
The resulting decision boundary is:

```

plot_svm_decision_boundary(w, b, sv_X, 4, 6, "Petal length", "Petal width")

plt.plot(X[:, 0][y==1], X[:, 1][y==1], "g^", label="Iris virginica")
plt.plot(X[:, 0][y==-1], X[:, 1][y==-1], "bs", label="Rest")
plt.legend(loc="upper left")
plt.axis([4, 5.9, 0, 2.8])
plt.show()

```



The soft margin classifier finds a decision boundary that **tolerates some violations**: several points from both classes fall inside the margin or on the wrong side, as visible from the many support vectors scattered across the margin region. This is the effect of a relatively small C , which prioritizes a wide margin over strict classification. We can experiment with a larger value of C to see how the boundary changes:

```

# Select a larger value of C
C = 100

# Solve the dual problem with the new value of C
result = minimize(
    lambda alpha: -dual_objective_soft(alpha),
    x0=np.zeros(X.shape[0]),
    method='SLSQP',
    bounds=[(0, C)] * X.shape[0], # alpha_i ≤ C only change
    constraints={'type': 'eq', 'fun': lambda a: a @ y},

```

```

    options={'ftol': 1e-9, 'maxiter': 1000}
)
alpha = result.x

# Compute the new weight vector and bias term
w = (alpha * y) @ X
sv_mask = alpha > 1e-5
sv_X, sv_y = X[sv_mask], y[sv_mask]
b = np.mean(sv_y - sv_X @ w)

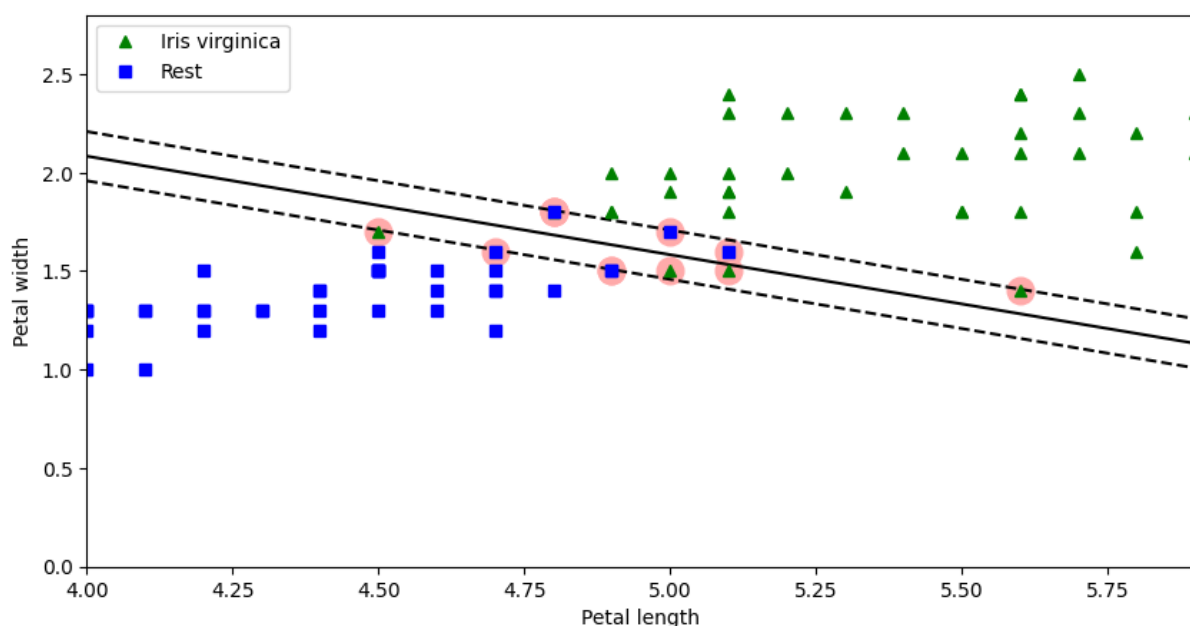
```

With the new value of C , the resulting decision boundary is:

```

plot_svm_decision_boundary(w, b, sv_X, 4, 6, "Petal length", "Petal width")
plt.plot(X[:, 0][y==1], X[:, 1][y==1], "g^", label="Iris virginica")
plt.plot(X[:, 0][y==-1], X[:, 1][y==-1], "bs", label="Rest")
plt.legend(loc="upper left")
plt.axis([4, 5.9, 0, 2.8])
plt.show()

```



The margin is much narrower and the number of support vectors has reduced significantly: the classifier is now **penalizing violations more heavily**, pushing the boundary closer to the overlapping region. The decision boundary has shifted and the model fits the training data more tightly, at the risk of being more sensitive to individual points near the boundary.

1.4 Nonlinear Classification

The optimal margin classifier we have developed so far assumes that the two classes can be separated by a hyperplane in the original feature space. When this assumption fails (and in practice it often does) the linear decision boundary is fundamentally inadequate, no matter how well the optimization is tuned. However, we have a natural way to extend the SVM framework to handle nonlinear problems: by **enriching the feature space**. In the chapter about Kernels methods, we saw that many learning algorithms can be reformulated entirely in terms of inner products between inputs, and that the **kernel trick** allows these inner products to be replaced by a kernel function that implicitly computes the inner product in a high-dimensional feature space, without ever constructing the feature mapping explicitly. The SVM dual objective is a natural instance of this structure. Looking back at the dual problem we derived:

$$\max_{\alpha} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)\top} x^{(j)}$$

the training data appears only through inner products $x^{(i)\top} x^{(j)}$. We can therefore apply the kernel trick directly: replace each inner product with a kernel evaluation:

$$x^{(i)\top} x^{(j)} \longrightarrow k(x^{(i)}, x^{(j)})$$

The dual problem becomes:

$$\max_{\alpha} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y^{(i)} y^{(j)} k(x^{(i)}, x^{(j)})$$

This single substitution turns the linear SVM into a **nonlinear one**, capable of **learning decision boundaries of arbitrary complexity** depending on the choice of kernel. The prediction for a new point also involves only inner products:

$$h(x) = \text{sign} \left(\sum_{i \in S} \alpha_i y^{(i)} k(x^{(i)}, x) + b \right)$$

where the sum runs over the support vectors only. The choice of kernel determines the geometry of the decision boundary in the original space. The only practical change in the implementation is that the matrix of inner products:

$$x^{(i)\top} x^{(j)}$$

is now replaced by a kernel matrix.

1.4.1 Kernels

As we saw in the kernel methods chapter, two kernels are particularly common in practice. The **polynomial kernel**:

$$k(x, x') = (x^\top x' + c)^d$$

implicitly maps the data into the space of all polynomial features up to degree d .

```
def polynomial_kernel(xi, xj, degree=3, coef=1):
    return (xi @ xj + coef) ** degree
```

The **RBF kernel**:

$$k(x, x') = \exp(-\gamma \|x - x'\|^2)$$

measures similarity through Euclidean distance and corresponds to an infinite-dimensional feature space.

```
def rbf_kernel(xi, xj, gamma=1):
    return np.exp(-gamma * np.linalg.norm(xi - xj) ** 2)
```

The kernel matrix collects all pairwise kernel evaluations between training points, playing exactly the role the inner product played in the linear case:

```
def build_kernel_matrix(X, kernel_fn):
    m = X.shape[0]
    K = np.zeros((m, m))
    for i in range(m):
        for j in range(m):
            K[i, j] = kernel_fn(X[i], X[j])
    return K
```

1.4.2 Kernelized SVM

The kernel matrix replaces the inner product in the dual objective without any other change to the optimization.

```
K = build_kernel_matrix(X, polynomial_kernel)

def dual_objective_kernel(alpha):
    return alpha.sum() - 0.5 * (alpha * y) @ K @ (alpha * y)
```

To demonstrate the kernelized SVM on a **nonlinearly separable dataset**, we use the **moons dataset**, two interleaving half-circles that no linear boundary can separate.

```

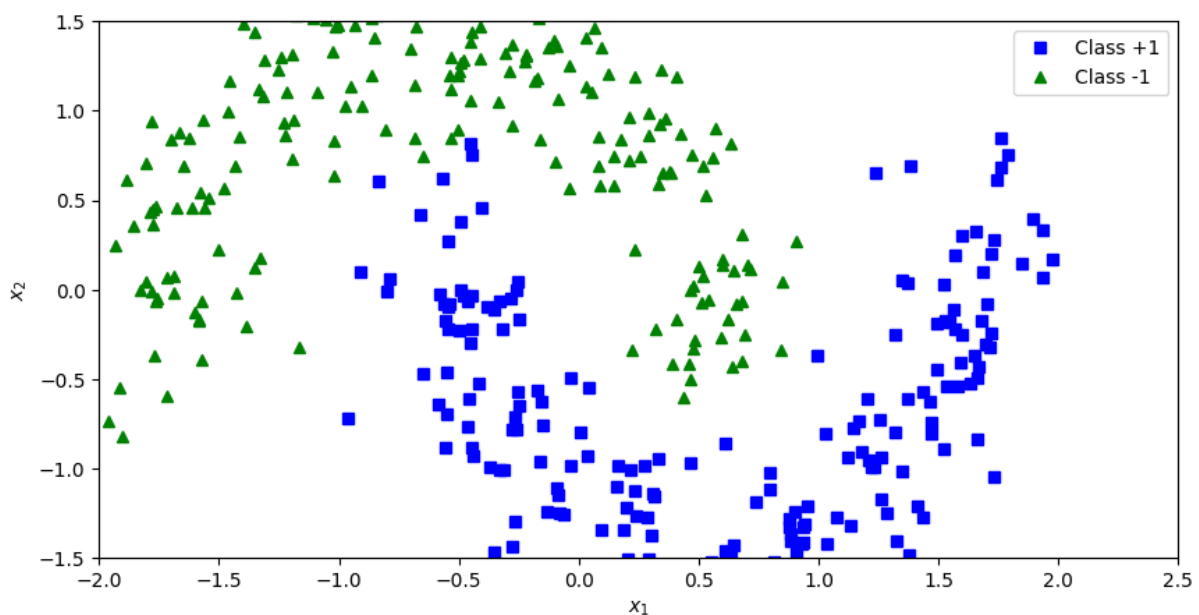
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler

# Create the moons dataset
X_moons, y_moons = make_moons(n_samples=400, noise=0.15, random_state=42)

# Standardize features before training, as the RBF kernel is sensitive to scale
scaler = StandardScaler()
X = scaler.fit_transform(X_moons)
y = np.where(y_moons == 0, -1, 1).astype(float)

# Plot the moons dataset
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(X[y==1, 0], X[y==1, 1], 'bs', label="Class +1")
ax.plot(X[y==-1, 0], X[y==-1, 1], 'g^', label="Class -1")
ax.set_xlabel("$x_1$")
ax.set_ylabel("$x_2$")
ax.legend(loc="upper right")
ax.axis([-2, 2.5, -1.5, 1.5]);

```



We now build the kernel matrix for the moons dataset and solve the dual problem with the polynomial kernel:

```

# Build the kernel matrix for the moons dataset
K = build_kernel_matrix(X, polynomial_kernel)

# Solve the dual problem with the polynomial kernel
result = minimize(
    lambda alpha: -dual_objective_kernel(alpha),
    x0=np.zeros(X.shape[0]),
    method='SLSQP',
    bounds=[(0, C)] * X.shape[0],
    constraints={'type': 'eq', 'fun': lambda a: a @ y},
    options={'ftol': 1e-9, 'maxiter': 1000}
)

# Extract the optimal alphas
alpha = result.x

```

After solving the dual problem, the solution is entirely encoded in the multipliers. Unlike the linear case, **there is no explicit weight vector w** to recover. It lives in the feature space induced by the kernel, which may be infinite-dimensional and is never constructed explicitly. This is not a limitation, it is precisely the point. Everything needed for prediction can be expressed in terms of kernel evaluations between the support vectors and the new input, without ever knowing w :

$$h(x) = \text{sign}\left(\sum_{i \in S} \alpha_i^* y^{(i)} k(x^{(i)}, x) + b\right)$$

The bias b is recovered using the same complementary slackness argument as before, replacing the linear inner product with the kernel:

$$b = \frac{1}{|S|} \sum_{i \in S} \left(y^{(i)} - \sum_{j \in S} \alpha_j^* y^{(j)} k(x^{(j)}, x^{(i)}) \right)$$

```

# Identify support vectors
sv_mask = alpha > 1e-5
sv_idx = np.where(sv_mask)[0]

# Compute the bias term using the support vectors
b = np.mean([
    y[i] - sum(alpha[j] * y[j] * polynomial_kernel(X[j], X[i]) for j in sv_idx)
    for i in sv_idx
])

# Predict function using the kernel

```

```
def predict_kernel(X_train, alpha, y, b, kernel_fn):
    sv_mask = alpha > 1e-5
    sv_X = X_train[sv_mask]
    sv_alpha = alpha[sv_mask]
    sv_y = y[sv_mask]
    def decision(x):
        return sum(sv_alpha[i] * sv_y[i] * kernel_fn(sv_X[i], x)
                    for i in range(len(sv_alpha))) + b
    return np.array([np.sign(decision(x)) for x in X_train])
```

To visualize the decision boundary we evaluate the decision function on a dense grid of points and draw the contour where it crosses zero. Unlike the linear case, the boundary has no closed-form expression in terms of w and b , so we compute it point by point using the kernel:

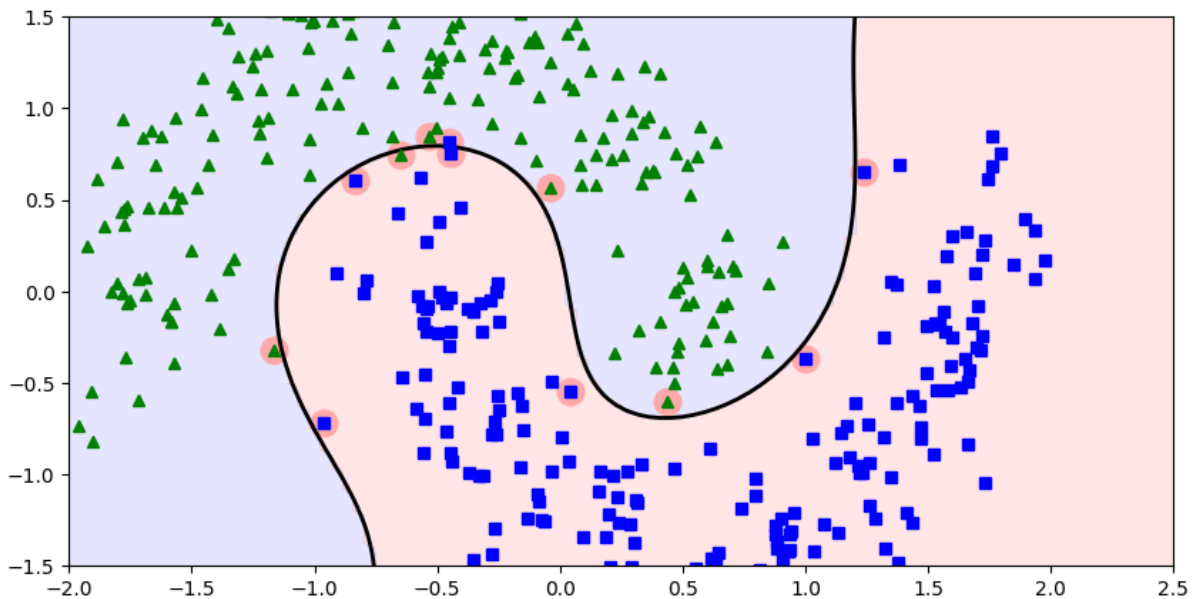
```
def plot_kernel_boundary(X, y, alpha, b, kernel_fn, axes):
    x0s = np.linspace(axes[0], axes[1], 100)
    x1s = np.linspace(axes[2], axes[3], 100)
    x0, x1 = np.meshgrid(x0s, x1s)
    grid = np.c_[x0.ravel(), x1.ravel()]

    sv_mask = alpha > 1e-5
    sv_X = X[sv_mask]
    sv_alpha = alpha[sv_mask]
    sv_y = y[sv_mask]

    scores = np.array([
        sum(sv_alpha[i] * sv_y[i] * kernel_fn(sv_X[i], x) for i in
        range(len(sv_alpha))) + b
        for x in grid
    ]).reshape(x0.shape)

    fig, ax = plt.subplots(figsize=(10, 5))
    ax.contourf(x0, x1, scores > 0, alpha=0.2, cmap='bwr')
    ax.contour(x0, x1, scores, levels=[0], colors='k', linewidths=2)
    ax.scatter(X[sv_mask, 0], X[sv_mask, 1], s=180, facecolors='#FFAAAA')
    ax.plot(X[y==1, 0], X[y==1, 1], 'bs')
    ax.plot(X[y==-1, 0], X[y==-1, 1], 'g^')
    ax.axis(axes)

plot_kernel_boundary(X, y, alpha, b, polynomial_kernel, axes=[-2, 2.5, -1.5, 1.5])
```



The polynomial kernel learns boundaries defined by polynomial curves. A more flexible alternative is the RBF kernel, which can produce boundaries of arbitrary shape. Replacing the kernel function is the only change required.

1.5 Regression

The SVM framework is not limited to classification. The same large-margin principle can be **adapted to regression**, but the objective is **inverted**: instead of finding a hyperplane that **separates** two classes with maximum margin, we look for a function that **fits** the training points while being as flat as possible. It is referred to as **Support Vector Regression (SVR)**. The key idea is to introduce a margin of tolerance around the true values, and only penalize predictions that fall outside this margin. This allows the model to ignore small errors and focus on fitting the overall trend of the data.

In classification, a point is either on the correct side of the margin or it is not, there is no notion of "how close" a correct prediction is. In regression, predictions are continuous, and **we need to define what counts as a good enough fit**. SVR introduces a **threshold epsilon**: predictions within this margin of the true value are considered perfect and incur no penalty at all. This **defines a tube** around the regression function, and only points that fall **outside** the tube are penalized.

This is the key difference from classification: the epsilon parameter is an explicit tolerance on the residuals, with no equivalent in the classification setting. Once a point is inside the tube, making the prediction even closer to the true value does not improve the objective, the model

is deliberately insensitive to small errors, which makes it robust to noise. Points outside the tube incur a penalty proportional to how far they exceed the margin, controlled by the same C parameter which plays the same role as in soft-margin classification.

1.5.1 Linear regression

We apply the same Lagrangian framework used for classification. The regression function is

$$\hat{y}^{(i)} = w^\top x^{(i)} + b$$

and a point is inside the tube if

$$|\hat{y}^{(i)} - y^{(i)}| \leq \varepsilon$$

Points outside the tube violate this condition in one of two ways: they either lie **above** the tube

$$y^{(i)} > \hat{y}^{(i)} + \varepsilon$$

or **below** it

$$y^{(i)} < \hat{y}^{(i)} - \varepsilon$$

The primal problem introduces one slack variable per point per direction and penalizes their sum:

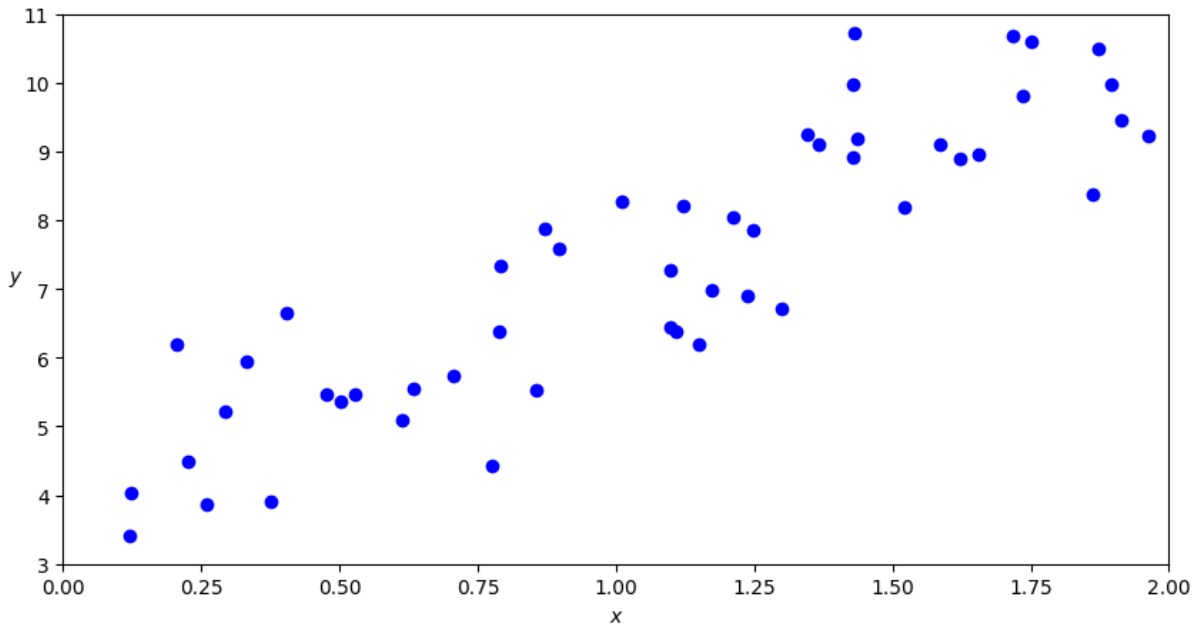
$$\begin{aligned} \min_{w, b, \zeta, \zeta^*} \quad & \frac{1}{2} w^\top w + C \sum_{i=1}^m (\zeta_i + \zeta_i^*) \\ \text{s.t.} \quad & y^{(i)} - w^\top x^{(i)} - b \leq \varepsilon + \zeta_i \quad \text{for } i = 1, \dots, m \\ & w^\top x^{(i)} + b - y^{(i)} \leq \varepsilon + \zeta_i^* \quad \text{for } i = 1, \dots, m \\ & \zeta_i, \zeta_i^* \geq 0 \end{aligned}$$

To illustrate, we generate a random linear dataset:

```
# Define the number of samples
m = 50

# Generate a random linear dataset
X = 2 * np.random.rand(m, 1)
y = (4 + 3 * X + np.random.randn(m, 1)).ravel()

# Plot the dataset
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(X, y, "bo")
ax.set_xlabel("$x$")
ax.set_ylabel("$y$", rotation=0)
ax.axis([0, 2, 3, 11]);
```



Rewriting the constraints in standard form (less than or equal to) and introducing one multiplier per constraint, the Lagrangian is:

$$\mathcal{L} = \frac{1}{2}w^\top w + C \sum_{i=1}^m (\zeta_i + \zeta_i^*) + \sum_{i=1}^m \alpha_i (y^{(i)} - w^\top x^{(i)} - b - \epsilon - \zeta_i) + \sum_{i=1}^m \alpha_i^* (w^\top x^{(i)} + b - y^{(i)} - \epsilon - \zeta_i^*) - \sum_{i=1}^m (\mu_i \zeta_i + \mu_i^* \zeta_i^*)$$

Applying the stationarity KKT conditions:

$$\nabla_w \mathcal{L} = w - \sum_{i=1}^m (\alpha_i - \alpha_i^*) x^{(i)} = 0 \quad \Rightarrow \quad w = \sum_{i=1}^m (\alpha_i - \alpha_i^*) x^{(i)}$$

$$\nabla_b \mathcal{L} = - \sum_{i=1}^m (\alpha_i - \alpha_i^*) = 0 \quad \Rightarrow \quad \sum_{i=1}^m (\alpha_i - \alpha_i^*) = 0$$

$$\frac{\partial \mathcal{L}}{\partial \zeta_i} = C - \alpha_i - \mu_i = 0 \quad \Rightarrow \quad 0 \leq \alpha_i \leq C$$

$$\frac{\partial \mathcal{L}}{\partial \zeta_i^*} = C - \alpha_i^* - \mu_i^* = 0 \quad \Rightarrow \quad 0 \leq \alpha_i^* \leq C$$

The last two conditions follow the same argument as in the soft margin case: the slack variables introduce an upper bound C on the multipliers. Substituting the expression for w back into the Lagrangian and simplifying yields the dual problem:

$$\begin{cases} \max_{\alpha, \alpha^*: 0 \leq \alpha_i, \alpha_i^* \leq C} \left(-\epsilon \sum_{i=1}^m (\alpha_i + \alpha_i^*) + \sum_{i=1}^m y^{(i)} (\alpha_i - \alpha_i^*) - \frac{1}{2} \sum_{i,j=1}^m (\alpha_i - \alpha_i^*) (\alpha_j - \alpha_j^*) x^{(i)\top} x^{(j)} \right) \\ \text{s.t.} \quad \sum_{i=1}^m (\alpha_i - \alpha_i^*) = 0 \end{cases}$$

The dual variables come in pairs and are concatenated into a single vector for the solver:

```
# Define the size of the tube (epsilon)
epsilon = 1.0

# Define the error penalty
C = 1.0

# Define the dual objective function
def svr_dual_objective(params):
    alpha      = params[:m]
    alpha_star = params[m:]
    diff = alpha - alpha_star
    return (epsilon * (alpha + alpha_star).sum() - y @ diff + 0.5 * diff @ X @ X.T @
    ↪ diff)
```

We solve the dual problem with the same numerical approach as before, concatenating the two sets of multipliers into a single vector and enforcing both the box constraints and the equality constraint:

```
result = minimize(
    svr_dual_objective,
    x0=np.zeros(2 * X.shape[0]),
    method='SLSQP',
    bounds=[(0, C)] * (2 * X.shape[0]),          # 0 ≤ alpha_i,
    ↪ alpha_i* ≤ C
    constraints={'type': 'eq',
                 'fun': lambda p: (p[:X.shape[0]] - p[X.shape[0]:]).sum()}, #
                 ↪ sum(alpha_i - alpha_i*) = 0
    options={'ftol': 1e-9, 'maxiter': 1000}
)

alpha      = result.x[:X.shape[0]]
alpha_star = result.x[X.shape[0]:]
```

From the stationarity condition:

$$w = \sum_{i=1}^m (\alpha_i - \alpha_i^*) x^{(i)}$$

we recover w directly from the dual variables. For b , the complementary slackness conditions state that a multiplier can only be nonzero if its corresponding constraint is active:

$$\alpha_i > 0 \Rightarrow y^{(i)} - w^\top x^{(i)} - b - \varepsilon = 0 \quad \Rightarrow \quad b = y^{(i)} - w^\top x^{(i)} - \varepsilon$$

$$\alpha_i^* > 0 \Rightarrow w^\top x^{(i)} + b - y^{(i)} - \varepsilon = 0 \quad \Rightarrow \quad b = y^{(i)} - w^\top x^{(i)} + \varepsilon$$

The first condition applies to support vectors **above** the tube, the second to support vectors **below** the tube. We average over all support vectors for numerical stability.

```
# Compute the weight vector w
diff = alpha - alpha_star
w = diff @ X

# Compute the bias term b
b_estimates = []

sv_above = alpha > 1e-5
sv_below = alpha_star > 1e-5

if sv_above.any():
    b_estimates.extend((y[sv_above] - X[sv_above] @ w - epsilon).tolist())

if sv_below.any():
    b_estimates.extend((y[sv_below] - X[sv_below] @ w + epsilon).tolist())

b = np.mean(b_estimates)
```

Once w and b are recovered, prediction reduces to a simple linear function:

```
def predict_svr(X_new, X_train, alpha, alpha_star, b, kernel_fn=None):
    if kernel_fn is None:
        kernel_fn = lambda xi, xj: xi @ xj
    diff = alpha - alpha_star
    sv_mask = np.abs(diff) > 1e-5
    sv_X = X_train[sv_mask]
    sv_diff = diff[sv_mask]
    def decision(x):
        return sum(sv_diff[i] * kernel_fn(sv_X[i], x)
                    for i in range(len(sv_diff))) + b
    return np.array([decision(x) for x in X_new])
```

We plot the regression line together with the tube, points inside the tube are ignored by the model, while those outside are the support vectors that determine w and b :

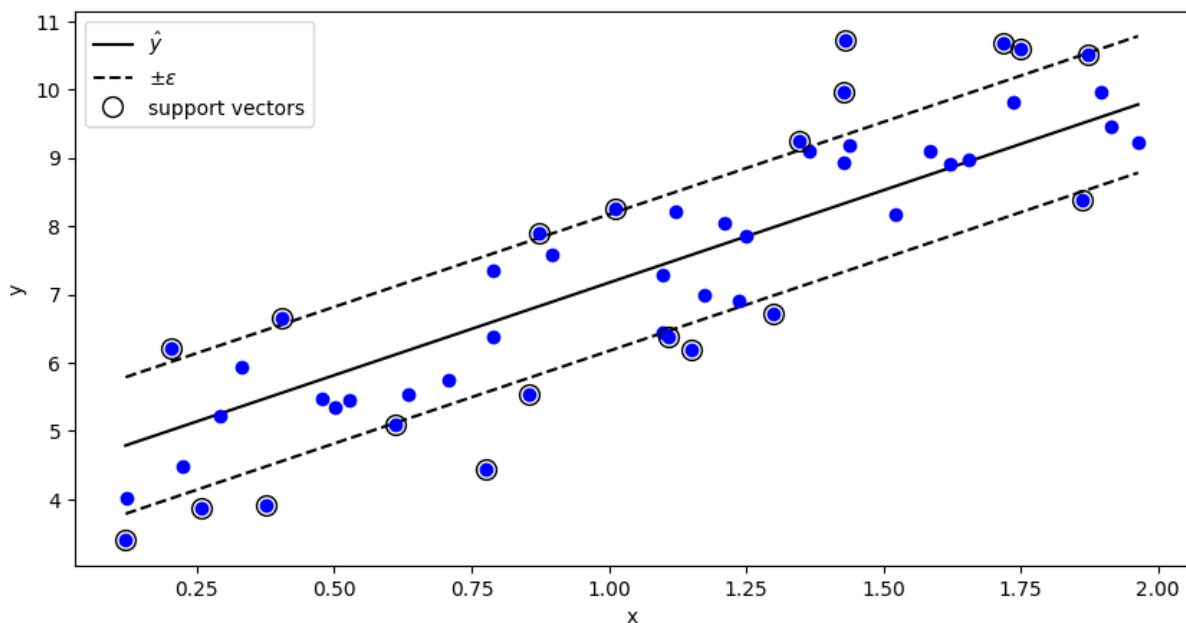
```

x_line = np.linspace(X.min(), X.max(), 200).reshape(-1, 1)
y_pred = predict_svr(x_line, X, alpha, alpha_star, b)

sv_mask = sv_above | sv_below

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(x_line, y_pred, "k-", label=r"$\hat{y}$")
ax.plot(x_line, y_pred + epsilon, "k--", label=r"$\pm\epsilon$")
ax.plot(x_line, y_pred - epsilon, "k--")
ax.plot(X, y, "bo")
ax.plot(X[sv_mask], y[sv_mask], "ko", markersize=10, fillstyle="none", label="support vectors")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.legend()
plt.show()

```



Epsilon controls the **width of the tube**. A larger epsilon produces a **wider tube**, which can improve generalization by ignoring more noise, but may underfit if taken too far.

```

epsilon = 0.5

result = minimize(
    svr_dual_objective,

```

```

x0=np.zeros(2 * X.shape[0]),
method="SLSQP",
bounds=[(0, C)] * (2 * X.shape[0]),
constraints={"type": "eq",
             "fun": lambda p: (p[:X.shape[0]] - p[X.shape[0]:]).sum()},
options={"ftol": 1e-9, "maxiter": 1000}
)

alpha      = result.x[:X.shape[0]]
alpha_star = result.x[X.shape[0]:]

diff = alpha - alpha_star
w = diff @ X

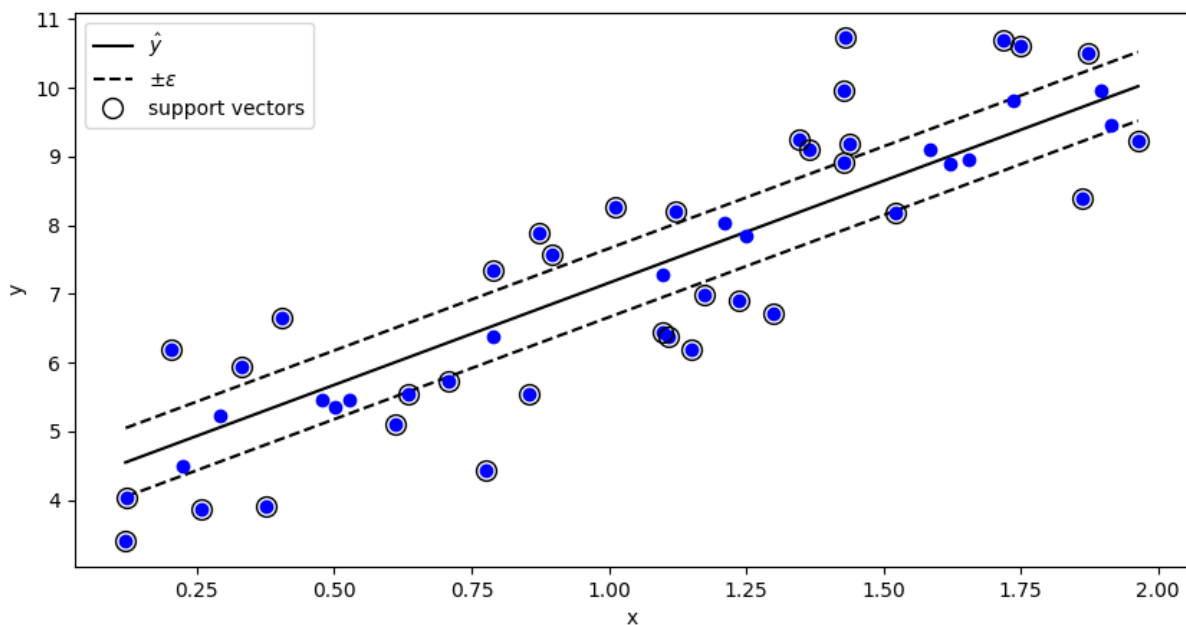
sv_above = alpha      > 1e-5
sv_below = alpha_star > 1e-5
sv_mask  = sv_above | sv_below

b_estimates = []
if sv_above.any():
    b_estimates.extend((y[sv_above] - X[sv_above] @ w - epsilon).tolist())
if sv_below.any():
    b_estimates.extend((y[sv_below] - X[sv_below] @ w + epsilon).tolist())
b = np.mean(b_estimates)

x_line = np.linspace(X.min(), X.max(), 200).reshape(-1, 1)
y_pred = predict_svr(x_line, X, alpha, alpha_star, b)

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(x_line, y_pred,          "k-",  label=r"$\hat{y}$")
ax.plot(x_line, y_pred + epsilon, "k--", label=r"$\pm \varpi$")
ax.plot(x_line, y_pred - epsilon, "k--")
ax.plot(X, y, "bo")
ax.plot(X[sv_mask], y[sv_mask], "ko", markersize=10, fillstyle="none", label="support  
↪ vectors")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.legend()
plt.show()

```



C controls the **penalty for points outside the tube**. A larger C penalizes violations more heavily, pushing the model to fit the training data more closely, at the risk of overfitting. A smaller C allows more violations, which can improve generalization but may underfit if taken too far.

```
C = 0.1

result = minimize(
    svr_dual_objective,
    x0=np.zeros(2 * X.shape[0]),
    method="SLSQP",
    bounds=[(0, C)] * (2 * X.shape[0]),
    constraints={"type": "eq",
                 "fun": lambda p: (p[:X.shape[0]] - p[X.shape[0]:]).sum()},
    options={"ftol": 1e-9, "maxiter": 1000}
)

alpha      = result.x[:X.shape[0]]
alpha_star = result.x[X.shape[0]:]

diff = alpha - alpha_star
w = diff @ X

sv_above = alpha > 1e-5
sv_below = alpha_star > 1e-5
sv_mask = sv_above | sv_below
```

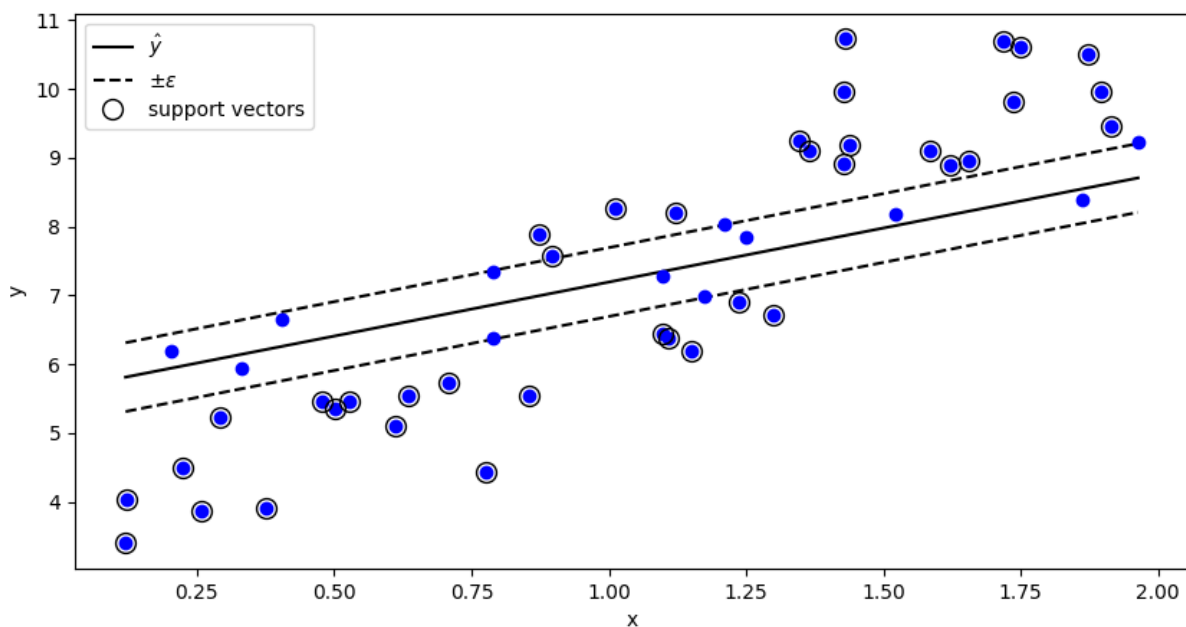
```

b_estimates = []
if sv_above.any():
    b_estimates.extend((y[sv_above] - X[sv_above] @ w - epsilon).tolist())
if sv_below.any():
    b_estimates.extend((y[sv_below] - X[sv_below] @ w + epsilon).tolist())
b = np.mean(b_estimates)

x_line = np.linspace(X.min(), X.max(), 200).reshape(-1, 1)
y_pred = predict_svr(x_line, X, alpha, alpha_star, b)

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(x_line, y_pred, "k-", label=r"$\hat{y}$")
ax.plot(x_line, y_pred + epsilon, "k--", label=r"$\pm \epsilon$")
ax.plot(x_line, y_pred - epsilon, "k--")
ax.plot(X, y, "bo")
ax.plot(X[sv_mask], y[sv_mask], "ko", markersize=10, fillstyle="none", label="support vectors")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.legend()
plt.show()

```



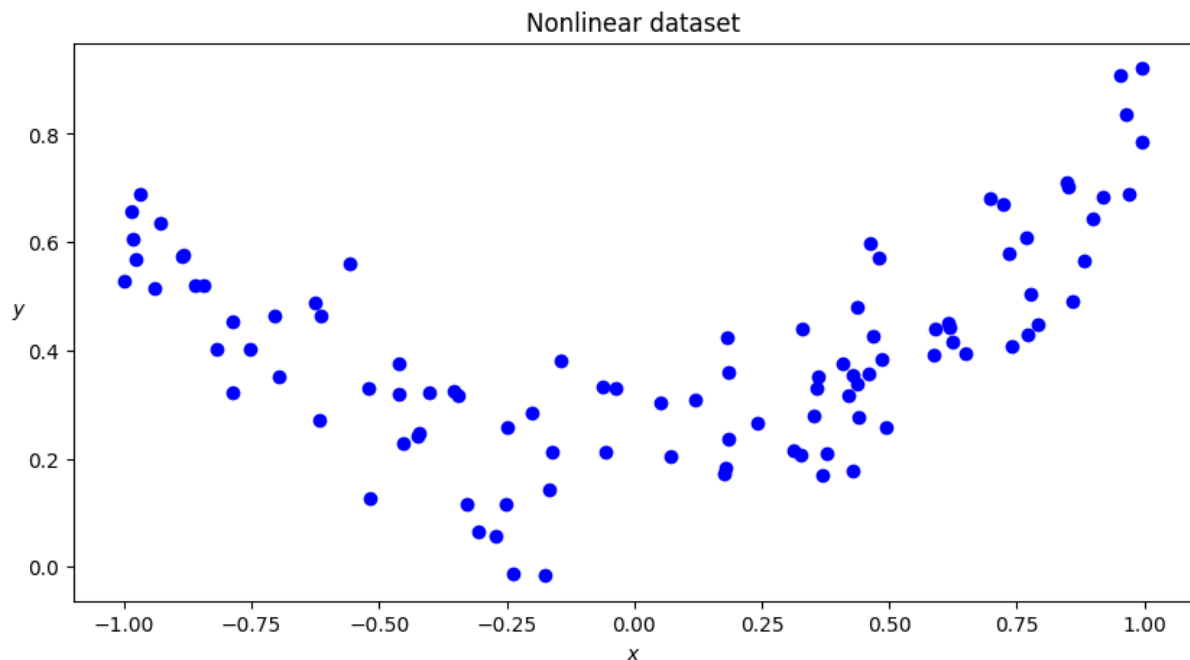
1.5.2 Kernelized Regression

Just as in classification, replacing the inner product with a kernel in the dual extends SVR to nonlinear regression without changing the optimization problem, only the kernel matrix needs to be built before solving. To illustrate, we generate a random nonlinear dataset:

```
# Define the number of samples
m = 100

# Generate a random nonlinear dataset
X = 2 * np.random.rand(m, 1) - 1
y = (0.2 + 0.1 * X + 0.5 * X**2 + np.random.randn(m, 1)/10).ravel()

# Plot the dataset
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(X, y, "bo")
ax.set_xlabel("$x$")
ax.set_ylabel("$y$", rotation=0)
ax.set_title("Nonlinear dataset")
plt.show()
```



We build the kernel matrix with a polynomial kernel and solve the dual problem as before, the only change from the linear case is that inne is replaced by K :

```

epsilon = 0.1
C = 100.0
m = X.shape[0]

K = build_kernel_matrix(X, lambda xi, xj: polynomial_kernel(xi, xj, degree=2))

def svr_dual_objective_kernel(params):
    alpha = params[:m]
    alpha_star = params[m:]
    diff = alpha - alpha_star
    return (epsilon * (alpha + alpha_star).sum()
            - y @ diff
            + 0.5 * diff @ K @ diff)

result = minimize(
    svr_dual_objective_kernel,
    x0=np.zeros(2 * m),
    method='SLSQP',
    bounds=[(0, C)] * (2 * m),
    constraints={'type': 'eq',
                 'fun': lambda p: (p[:m] - p[m:]).sum()},
    options={'ftol': 1e-9, 'maxiter': 1000}
)

alpha = result.x[:m]
alpha_star = result.x[m:]

```

Unlike the linear case, where w and b are explicitly computed from the dual variables, in the kernelized case there is no weight vector to recover. Prediction is made directly from the dual variables, which encode the solution entirely through the support vectors:

```

diff = alpha - alpha_star
sv_mask = np.abs(diff) > 1e-5

b_estimates = []
for i in np.where(sv_mask)[0]:
    k_i = np.array([polynomial_kernel(X[j], X[i], degree=2) for j in range(m)])
    if alpha[i] > 1e-5:
        b_estimates.append(y[i] - diff @ k_i - epsilon)
    else:
        b_estimates.append(y[i] - diff @ k_i + epsilon)
b = np.mean(b_estimates)

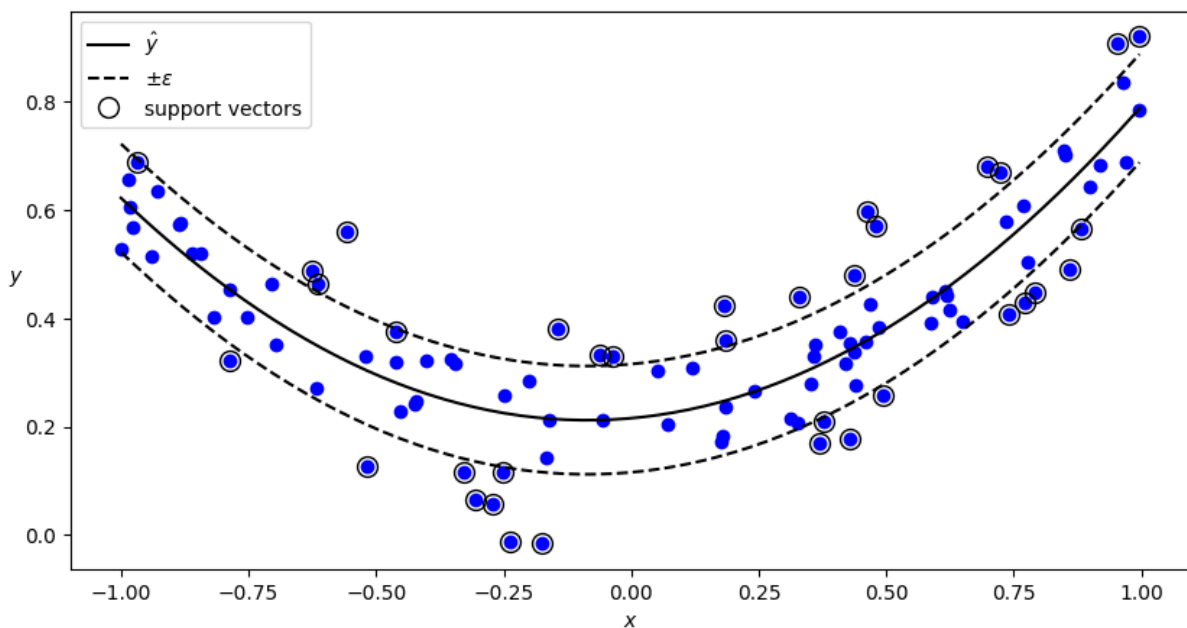
```

```

x_line = np.linspace(X.min(), X.max(), 200).reshape(-1, 1)
y_pred = predict_svr(x_line, X, alpha, alpha_star, b,
                    kernel_fn=lambda xi, xj: polynomial_kernel(xi, xj, degree=2))

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(X, y, "bo")
ax.plot(x_line, y_pred, "k-", label=r"$\hat{y}$")
ax.plot(x_line, y_pred + epsilon, "k--", label=r"$\pm \epsilon$")
ax.plot(x_line, y_pred - epsilon, "k--")
ax.plot(X[sv_mask], y[sv_mask], "ko", markersize=10, fillstyle='none', label="support vectors")
ax.set_xlabel("$x$"); ax.set_ylabel("$y$", rotation=0)
ax.legend()
plt.show()

```



1.6 Curse of dimensionality

Throughout this chapter we have seen that SVMs are determined entirely by the support vectors, the subset of training points that lie on or violate the margin. This is both a strength and a limitation. In low-dimensional settings, only a small fraction of the data typically lies near the decision boundary, so the number of support vectors remains limited. In this regime, the model is compact, efficient, and tends to generalize well. As the dimensionality increases, however,

the geometry of the margin changes. Adding new features introduces new directions in which points can approach or cross the boundary. As a result, points that were previously well inside the margin may become critical, and **the number of support vectors can grow significantly**. In high-dimensional spaces, it is not uncommon for a large portion of the dataset (in extreme cases, almost all points) to become support vectors.

This leads to an important practical guideline. When **the number of features is large but the number of training examples is relatively small**, SVMs are particularly effective. In this setting, the margin is well defined and only a limited number of points constrain the solution, leading to a compact model with good generalization properties. This is a common situation in fields such as bioinformatics, where datasets often contain few samples but thousands of features.

On the other hand, when **the number of training examples becomes large**, SVMs can become computationally demanding. The dual formulation relies on the kernel matrix, whose size grows quadratically with the number of data points, making both memory usage and optimization increasingly expensive. In such cases, alternative methods such as stochastic gradient techniques or neural networks are often more practical.

The most challenging scenario arises when both **the number of features and the number of examples are large**. In this case, the kernel matrix becomes prohibitively large, and the number of support vectors can also grow substantially, making the model both expensive to train and to evaluate.

1.6.1 Generalization bounds

One of the reasons SVMs work so well is that their ability to generalize can be understood through two complementary bounds: one theoretical, based on capacity, and one practical, based directly on the learned solution. The first is the **margin bound**, which links generalization to the geometry of the classifier. Consider all hyperplanes that separate the data with a margin at least γ , assuming the data lies inside a ball of radius R . The VC dimension h of this family satisfies:

$$h \leq \min\left(\left\lceil \frac{R^2}{\gamma^2} \right\rceil, n\right) + 1$$

This result has a very important interpretation. The VC dimension measures the capacity of the model, that is, how complex the set of classifiers is and how easily it can overfit the data. The bound shows that this capacity is controlled not only by the number of features n , but also and crucially by the ratio:

$$\frac{R^2}{\gamma^2}$$

The key insight is that increasing the margin γ reduces this ratio, and therefore reduces the capacity of the classifier. In other words, **a larger margin forces the classifier to be simpler**, even in high-dimensional spaces. This explains why SVMs can generalize well even when the number of features is large: what really matters is not the dimension, but how well-separated the data is.

However, this bound is mainly of conceptual value. In practice it tends to be quite loose, and does not directly predict the actual test error. A second, more practical bound is the **leave-one-out bound**, which exploits a key structural property of SVMs: only support vectors influence the solution. The leave-one-out error satisfies:

$$L_{\text{loo}} \leq \frac{\mathbb{E}[\text{number of support vectors}]}{m}$$

This bound is much more directly interpretable. It tells us that the generalization error is controlled by **how many training points are "critical"** for defining the decision boundary. If only a small fraction of the data are support vectors, then removing one point is unlikely to change the classifier, and the error will be small. The intuition is simple: if a point is not a support vector, removing it does not affect the learned hyperplane at all, so it cannot become misclassified. Only support vectors can potentially change the solution when removed. As a result, the leave-one-out error is essentially bounded by the fraction of support vectors in the dataset. This bound is also computationally attractive: it can be evaluated without retraining the model multiple times, since the set of support vectors is already known after training.

Together, these two bounds provide a clear picture: **the margin controls the capacity of the model**, while **the number of support vectors reflects its effective complexity in practice**.

1.6.2 Advanced techniques

The dual formulation of SVM leads to a quadratic optimization problem with m variables (one per training point) and m constraints. In principle, this can be solved with a generic quadratic programming solver. In practice, however, this **quickly becomes infeasible when the number of training examples grows large**. The main issue is not only **computational time**, but also **memory**: storing the full kernel matrix requires a huge amount of memory, which becomes prohibitive for large datasets. Moreover, the presence of a constraint for each training point means that the problem scales poorly as m increases. For datasets with tens or hundreds of thousands of examples, a standard solver is no longer practical. To overcome this limitation, SVMs are typically trained using the **Sequential Minimal Optimization (SMO)** algorithm. The idea behind SMO is to exploit the special structure of the dual problem and solve it in a much more efficient way. A natural approach would be to update one multiplier at a time, as in coordinate descent. However, this is not possible because the multipliers must satisfy the equality constraint

$$\sum_{i=1}^m \alpha_i y^{(i)} = 0$$

Updating a single variable would generally violate this constraint. SMO resolves this issue by updating the smallest possible set of variables that preserves feasibility. The algorithm proceeds iteratively as follows. At each step, it selects a multiplier that most strongly violates the optimality conditions, then chooses a second multiplier using an heuristic (typically the one that allows the largest possible update). The resulting two-variable subproblem is solved analytically, and the values of the two variables are updated accordingly. By repeating this process, the algorithm gradually improves the solution until all optimality conditions are satisfied. A key advantage of SMO is that it does not require storing the full kernel matrix; instead, it computes only the necessary entries on demand. This makes it scalable to much larger datasets than generic solvers. In practice, SMO converges to the global optimum of the SVM problem and is the algorithm used in widely adopted implementations such as `sklearn.svm.SVC` (via `LIBSVM`).

1.7 Exercise

Train an SVM classifier on the MNIST dataset. Since SVM classifiers are binary classifiers, we will need to use one-versus-the-rest to classify all 10 digits. Tune the hyperparameters using a small validation set to speed up the process. What accuracy can we reach?

We start by loading MNIST and splitting it into training and test sets:

```
from sklearn.datasets import fetch_openml
mnist = fetch_openml('mnist_784', version=1, cache=True, parser='auto')

X = mnist["data"]
y = mnist["target"].astype(np.uint8)

X_train = X[:60000]
y_train = y[:60000]
X_test = X[60000:]
y_test = y[60000:]
```

Since SVM optimization is sensitive to feature scale, we standardize the pixel values before training:

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train.astype(np.float32))
X_test_scaled = scaler.transform(X_test.astype(np.float32))
```

The from-scratch solver we built throughout this chapter works well on small datasets, but it cannot scale to MNIST. The dual problem requires storing and solving a big kernel matrix: with 60k training samples that is a ~28 GB matrix, and the solver used would need to optimize over 60k variables simultaneously, far beyond what a general-purpose optimizer can handle in practice. Production SVM solvers such as LIBSVM (used internally by scikit-learn's) address this with the SMO algorithm. We start with a **linear SVM** as a fast baseline, moreover, Scikit-learn handles multiclass classification internally, no manual one-versus-the-rest or one-versus-one scheme is needed:

```
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score

lin_svm = LinearSVC(C=1.0, dual='auto', max_iter=5000, random_state=42)
lin_svm.fit(X_train_scaled, y_train)

print('Train accuracy:', accuracy_score(y_train, lin_svm.predict(X_train_scaled)))
print('Test accuracy:', accuracy_score(y_test, lin_svm.predict(X_test_scaled)))
```

```
Train accuracy: 0.9275333333333333
```

```
Test accuracy: 0.9158
```

The linear model gives a reasonable baseline but MNIST digits are not linearly separable in pixel space. An RBF kernel SVM can learn non-linear boundaries and typically achieves much higher accuracy. Because training SVC on 60k samples is slow, we first tune hyperparameters on a small random subset using a randomized search cross-validation:

```
from sklearn.svm import SVC
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform
import warnings

warnings.filterwarnings('ignore')

# Use a small subset for fast hyperparameter search
X_search = X_train_scaled[:10000]
```

```
y_search = y_train[:10000]

param_distributions = {
    'C':      loguniform(0.1, 100),
    'gamma': loguniform(1e-4, 1e-1),
}

rnd_search = RandomizedSearchCV(
    SVC(kernel='rbf'),
    param_distributions,
    n_iter=10,
    cv=3,
    scoring='accuracy',
    random_state=42,
    verbose=0
)
rnd_search.fit(X_search, y_search)

print('Best params:', rnd_search.best_params_)
print('Best CV accuracy:', rnd_search.best_score_)
```

```
Best params: {'C': np.float64(31.428808908401084), 'gamma': np.float64(0.0004335281794951569)
Best CV accuracy: 0.9356000387081299
```

With the best hyperparameters found, we retrain on the full training set and evaluate on the test set:

```
best_svm = rnd_search.best_estimator_
best_svm.fit(X_train_scaled, y_train)

print('Train accuracy:', accuracy_score(y_train, best_svm.predict(X_train_scaled)))
print('Test accuracy:', accuracy_score(y_test, best_svm.predict(X_test_scaled)))
```

```
Train accuracy: 0.9973166666666666
Test accuracy: 0.9726
```

The final test accuracy with a well-tuned RBF kernel SVM typically reaches above 98% on MNIST, competitive with shallow neural networks.